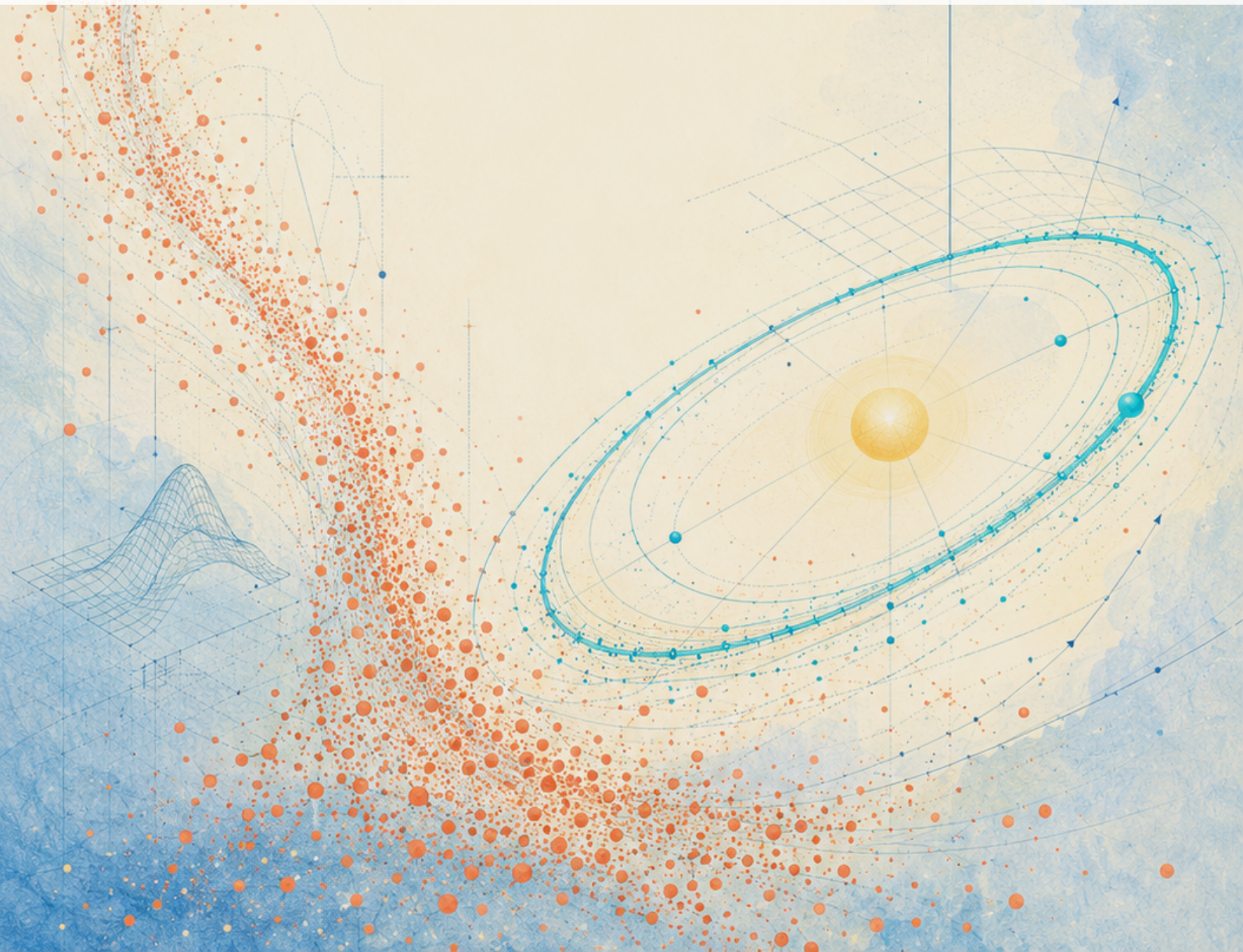
Physics Programming

Scientific Computing with Julia and Python

77641



Contents

-1 About the Course and These Notes . . . 1
-1.1 A map of the year . . . 1
-1.2 The materials that travel with each week . . . 1
-1.3 How to read the notes . . . 2
-1.4 Why is this Chapter -1? . . . 2

0 Intro: Physics in the Age of Computation . . . 3
0.1 Why Use Computers to Do Physics? . . . 3
0.1.1 Start with measurements and motion . . . 3
0.1.2 A star disrupted by a black hole . . . 3
0.1.3 Following a Higgs candidate at CERN . . . 4
0.2 What Is a Program? What Is a Computer? . . . 5
0.3 Why Start with Julia? . . . 5
0.4 Why Python? . . . 7
0.4.1 A complete example: the signal at both LIGO sites . . . 8
0.5 Why Learn Both? . . . 9
0.6 How the Year Is Organized . . . 10
0.7 What We Will Learn . . . 10
0.8 Teaser: Repeating Kepler's Mars Analysis . . . 10
0.8.1 Obtaining the course repository . . . 11
0.9 The course environment: install once, run reproducibly . . . 11

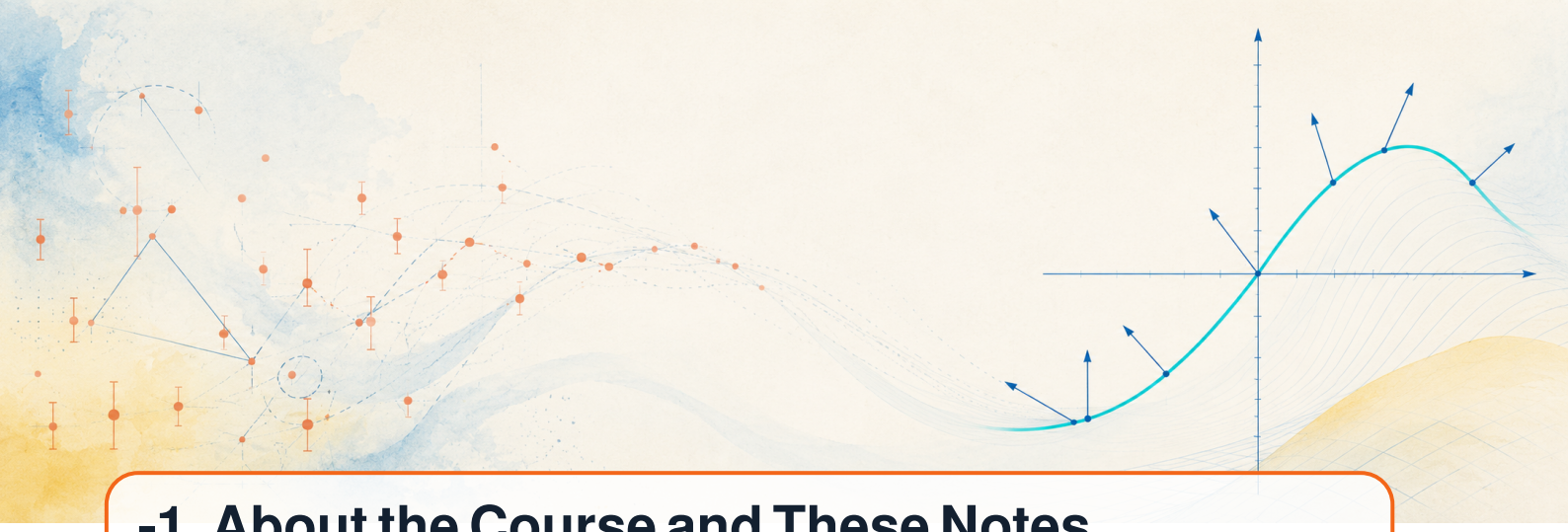
1 First Julia Calculations and a Guided Ohm-Law Fit . . . 13
1.1 Where do we see a result? . . . 13
1.2 Julia as a calculator . . . 13
1.3 Names for values before physics . . . 14
1.4 A guided measurement-to-model workflow . . . 14
1.4.1 Build the analysis one visible step at a time . . . 16
1.5 Recitation: measuring a spring constant . . . 21
1.6 Core homework: infer gravity from free-fall measurements . . . 22
1.7 Optional bonus: make a scientific figure communicate . . . 22
1.8 Going further: Lissajous curves . . . 23

2 Week 2: Values, Decisions, and a Block on an Incline . . . 25
2.1 Lecture: From Variables to an Algorithm . . . 25
2.1.1 A variable gives a value a name . . . 25
2.1.2 Julia follows the program in order . . . 26
2.1.3 A variable can be assigned a new value . . . 26
2.1.4 Values have types . . . 26

2.1.5 Four simple value types 27
2.1.6 Comparisons produce Boolean values 27
2.1.7 A first decision with if 28
2.2 Piecewise Kinematics as a Decision 29
2.2.1 Several branches 29
2.3 Values have types 30
2.4 Booleans and gradual decisions 31
2.5 Angles, logic, and validation before a calculation 32
2.6 A piecewise friction model on an incline 33
2.7 Recitation: a pull can reverse impending motion 34
2.8 Core homework: does a projectile clear a wall? 34
2.9 Optional bonus: units and AC phasors 34
2.10 Going further: total internal reflection 35

3 Vectors, Loops, and Sampled Motion 37
3.1 Vectors store ordered histories 37
3.1.1 Creating vectors 37
3.1.2 Indexing and slicing 38
3.1.3 Changing a vector 39
3.1.4 Vector arithmetic and useful functions 39
3.2 A planned repetition: the `for` loop 40
3.3 Summaries: mean and sample standard deviation 43
3.4 A condition-controlled repetition: `while` 43
3.5 From the derivative to an Euler update 44
3.6 Refinement: compare with a known answer 46
3.7 Recitation: first sampled cooling time 47
3.8 Core homework: a hammer throw with air resistance 47
3.9 Optional bonus: three integration schemes 48
3.10 Going further: from one walk to diffusion 48

4 Matrices, Loops, and Rotations 49
4.1 Matrix literals and shape 49
4.1.1 Indexing a matrix 49
4.2 while loops: repeat until a condition turns false 50
4.2.1 Sentinel input: a class average 50
4.2.2 Validating input 51
4.2.3 A guessing game 51
4.2.4 Timing a while loop: the harmonic series 52
4.3 Matrix–vector and matrix–matrix products 53
4.4 Nested loops: a multiplication table 54
4.5 Nested loops: matrix multiplication 54
4.6 Checking whether a matrix is symmetric 55
4.7 Rotation matrices in 2D 57
4.7.1 Checking that a matrix really is a rotation 57
4.8 Rotating a rectangle in the plane 58



-1. About the Course and These Notes

Why learn programming and use computers in physics? We need computers because there is just so much we can do with lab experiments, a pencil and paper. We need to analyze lab data, we need to solve models which do not have analytical solutions, we want to simulate reality, automate experiments, and make our lives as physicists much easier. Throughout this course, we will use computation to connect ideas from physics and mathematics with data, numerical methods, and the scientific questions we want to answer.

-1.1 A map of the year

We begin with Julia and the small, understandable calculations that help us build confidence: reading a supplied table, plotting measurements, estimating a straight-line fit, and tracing simple motion. From there, we will write our own algorithms for numerical methods, compare them with established packages, and combine them in larger investigations. Later we will use Python to carry familiar scientific ideas into a second language and its broad research ecosystem. Chapter 0 introduces the subject; from Chapter 1 onward, each chapter number matches the course week.

-1.2 The materials that travel with each week

Each week's materials have a distinct role:

- **Lecture book, Jupyter notebooks and Slides:** this book develops the ideas with worked examples and their outputs; the Jupyter notebooks follows the same path in executable form, so we can run and explore those examples interactively. Opening a notebook is optional, and can be done in VS Code with the Jupyter extension or in JupyterLab; the notebooks also carry saved outputs. The slides are abridged versions (with less explanations) but they support the live lecture.
- **Recitation guide and notebook:** Each chapter begins with a brief recap and one short worked example, then spend most of the recitation working on the core homework with help from the teaching staff. The homework assignment itself is a separate document.
- **Core homework:** the weekly homework gives us practice with the material, and a worked solution supports teaching and review. We submit it on the course Moodle

page, where the assignment and its submission link live. The first homework is graded by hand in Week 1: we upload a script and a figure. From Week 2, CodeRunner grades suitable programming questions automatically by running published tests against our submission; the question describes what to provide and what the tests check.

- **Optional study:** each completed bonus is worth one-third **extra** point to the final grade. The bonus is optional, is not examined, and does not introduce material needed in a later week. A separate *Going further* section offers optional self-study beyond the course core, while weekly self-study questions help us check our understanding before an exam.

-1.3 How to read the notes

Colored boxes give us a place to pause over an idea that deserves its own explanation:

- **Gray-blue computing boxes** explain programming tools, conventions, or workflow.
- **Purple physics boxes** set out the physical model, its quantities, and its limits. This is material not officially part of the course, but it places our example in the relevant physics context.
- **Green mathematics boxes** develop a mathematical relation or derivation. Like the physics boxes, they are not an official part of the course, but allows links our example to the math we concurrently learn.
- **Orange boxes** offer a hint, a warning, or a useful point to keep in mind.

Code appears in pale-blue listings. The pale-green listing immediately below a code example shows its output. We can compare that output with the explanation that follows, then use the same habit when we run a notebook ourselves.

-1.4 Why is this Chapter -1?

This short orientation (chapter -1) comes before the introduction to physics programming (chapter 0), while the course itself starts with chapter 1, which corresponds to week 1. Thus, this designation allows alignment of the course weeks and chapters together.

0. Intro: Physics in the Age of Computation

0.1 Why Use Computers to Do Physics?

Why do we need programming and computers in physics? They help us follow an idea from a model to a calculation, and from a calculation to measurements we can inspect. We can start with familiar questions and then see how the same approach reaches problems at the frontiers of research.

0.1.1 Start with measurements and motion

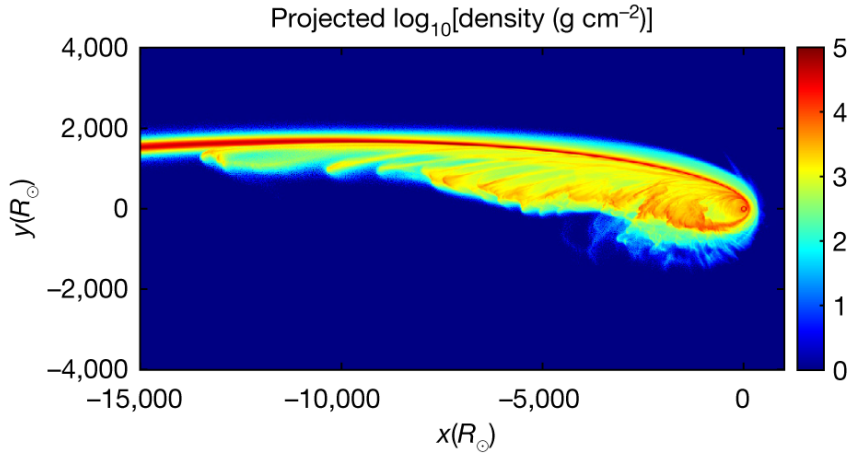
Suppose we measure voltage and current for a conductor. A plot lets us see how the two quantities change together; a straight-line fit gives us an estimate of resistance when the conductor behaves ohmically. In Week 1, we will follow a supplied measurement-to-plot-to-fit example. Later, we will build the fitting method ourselves and examine how the residuals and measurement uncertainty affect the conclusion.

In mechanics, a simple projectile model leads to a parabolic trajectory that we can calculate with familiar equations. What changes when air resistance depends on velocity, or when air density varies with height? Adding these effects couples position and velocity in the equations of motion. Numerical integration advances them in small time steps, letting us compare the ideal path with a more detailed prediction.

0.1.2 A star disrupted by a black hole

What happens when a star passes too close to a massive black hole? The black hole pulls more strongly on the near side of the star than on the far side. When that tidal difference overcomes the star's self-gravity, the star is disrupted. Its gas stretches into streams that return, collide, heat up, and begin to form an accretion flow.

Analytical estimates tell us where disruption begins and set characteristic orbital timescales. To follow the returning streams as they collide, form shocks, and radiate, researchers solve coupled hydrodynamic and radiation equations numerically across three-dimensional space and time. The image below shows the resulting projected column density of such a calculation carried out by two members of our department; its color scale traces where the disrupted gas is concentrated. Such simulations also predict how the event's light changes as the flow develops and allows us to make predictions completely impossible with analytic calculations.

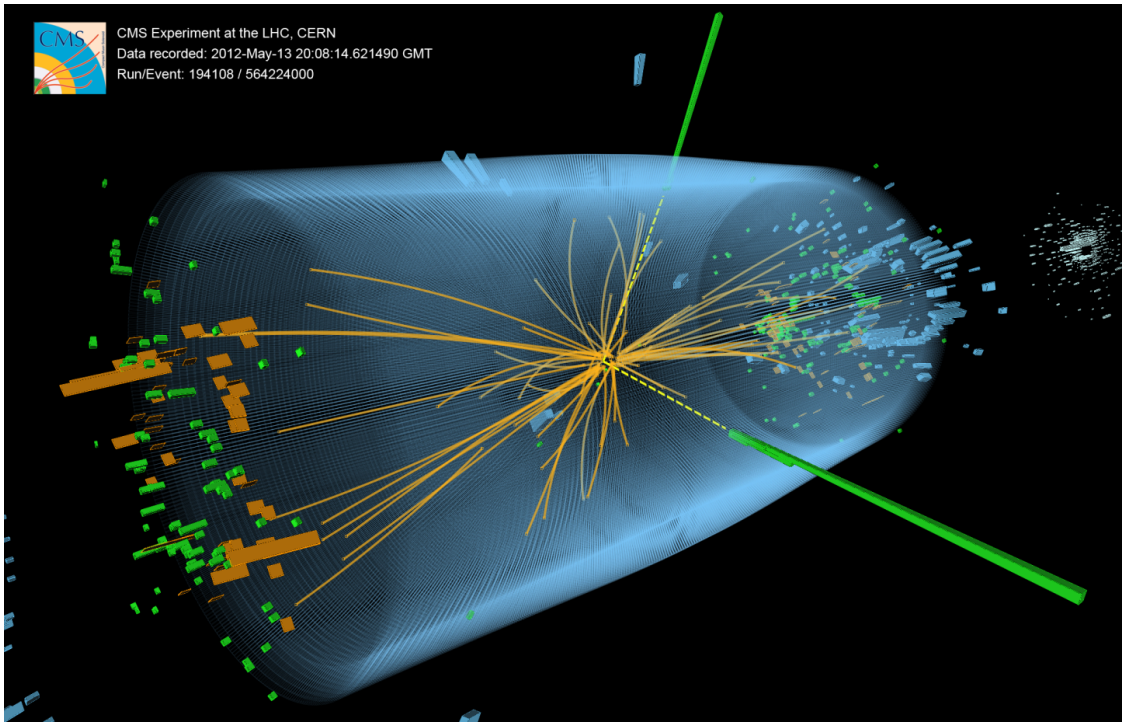


The projected log column density during a simulated tidal disruption event; positions are measured in solar radii. From E. Steinberg and N. C. Stone, "Stream–disk shocks as the origins of peak light in tidal disruption events," Nature 625, 463–467 (2024), Fig. 1, doi:10.1038/s41586-023-06875-y.

0.1.3 Following a Higgs candidate at CERN

At CERN, the Higgs boson is created in very rare particle collisions and decays almost immediately. The detectors at CERN record millions of collision events. For each one they have to calculate the dynamics of all the particles involved from the measured energies and directions of the particles produced. This is a completely impossible task for normal humans, which is why computerized analysis is necessary.

The image below shows one particular event which produced two photons, and which is a very likely candidate for the formation and decay of the Higgs particle. This particle was theorized to exist in order to explain how some particles received their mass.



A CMS diphoton candidate from the Higgs search, drawn from reconstructed detector signals. Image: CMS/CERN, CERN Higgs boson overview.

These two examples show us how computers can be used to analyze data and model systems in ways that are qualitatively and quantitatively different, offering more tools for us physicsts.

0.2 What Is a Program? What Is a Computer?

A *program* is a sequence of instructions that tells a computer how to carry out a task. For our projectile, it could start with a position and velocity, calculate the acceleration, and update the motion at successive times. To write those instructions, we need to say which data to store, how to calculate with them, and when to repeat or stop. A human reader can often fill in a missing step in a derivation; a computer needs that step spelled out.

A *computer* is a physical machine that repeatedly executes these instructions. Its central processor (CPU) carries out arithmetic and logical operations; memory (RAM) holds the data and instructions currently in use; persistent storage keeps files after the program ends; and input/output devices connect the machine to files, screens, instruments, and networks. When we run a Julia script, the Julia language system translates our instructions into operations the CPU can execute, while the operating system manages files, memory, and access to the screen.

As we read a program, we can follow the data from the starting values through the calculations to the result. An *algorithm* describes the steps of that calculation; the physical model helps us interpret what comes out. We will build up this way of working gradually. Week 2 introduces types, input conversion and decisions; Week 3 introduces loops and stopping rules; Week 5 introduces function definitions.

0.3 Why Start with Julia?

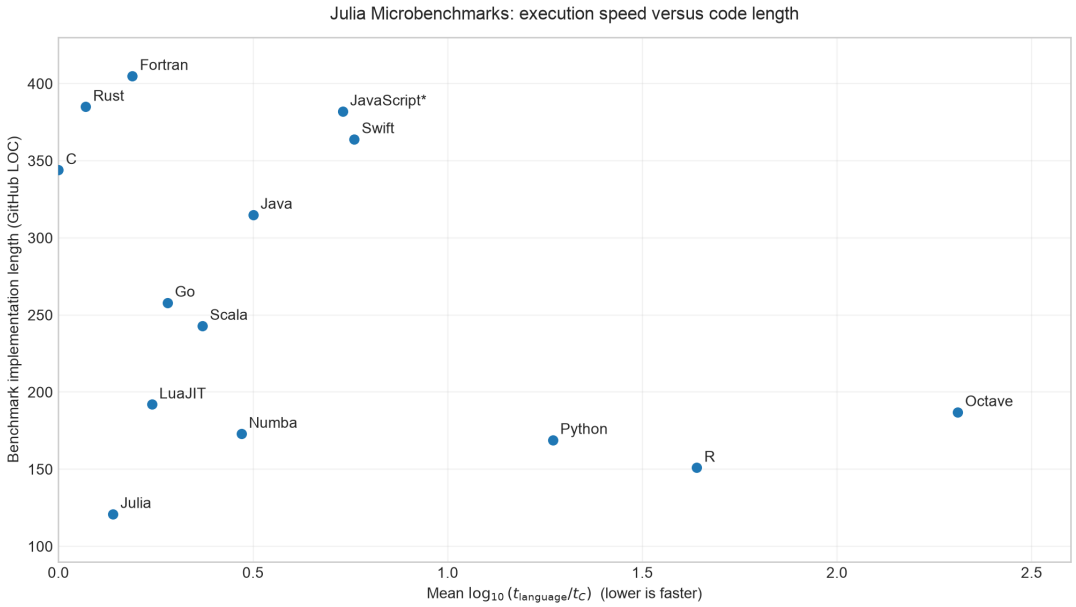
Julia was designed for numerical and scientific work. It has a direct notation for vectors, matrices, linear algebra, multiple dispatch, and differential-equation solvers, while still being suitable for exploratory scripts and notebooks. It is also fast enough that the first readable version of a numerical model is often already useful.

For us as physicists, the important first benefit is that the code can look like the model. A vector of measurements, a matrix that represents a linear transformation, and an array of values sampled in time are ordinary Julia objects. The distinction between a single value, a vector, and an element-by-element calculation is visible in the notation. That makes dimensions, indices, units, and the intended physical operation easier to inspect before a numerical mistake becomes buried in a long program.

Julia is expressive because its notation lets us state an operation close to the mathematics, and composable because well-defined operations and package tools can be combined into a larger calculation. A matrix operation can feed into another calculation, for example, while the same numerical idea can be defined for ordinary numbers, vectors, matrices, or later for values that carry units. Julia selects the appropriate method from the types of the inputs; this is called *multiple dispatch*. We will use scientific tools first and return to this design idea when it helps explain how one operation can work across several physical settings.

Julia projects can record the packages and versions used for an analysis in project files. This makes it practical to return to a calculation months later or share it with a collaborator without silently changing the computational tools underneath it. The language also has a strong standard library for numerical linear algebra and a growing ecosystem for differential equations, optimization, statistics, units, plotting, machine learning and scientific data. We will build the core methods ourselves first, then use those packages as carefully checked implementations of ideas we already understand.

Speed matters in physics because it lets us test a model, inspect a result, change an assumption, and repeat. The figure below summarizes the Julia project's microbenchmark suite. Its horizontal axis compares execution time with C on a logarithmic scale, so points further left are faster; its vertical axis is the length of the benchmark implementation. Julia lies close to compiled languages in these small numerical kernels while its example code remains compact. These timings describe the selected kernels; the speed of a full program also depends on its algorithm, data layout, allocations, and how we measure it. The data and programs are available in the Julia microbenchmark repository.



Most importantly for this course, Julia allows code to follow the mathematical structure closely. Suppose a linear dynamical model is written as

$$\mathbf{x}_{n+1} = A\mathbf{x}_n + B\mathbf{u}_n, \qquad \mathbf{y}_n = C\mathbf{x}_n.$$

The Julia implementation is almost identical:

```
x_next = A * x + B * u
y = C * x
```

Here `x`, `u`, and `y` are vectors, while `A`, `B`, and `C` are matrices. We can compare each line of code directly with its equation. As we do, checking the matrix dimensions and physical units helps us see whether the calculation expresses the model we intended.

There are practical details to get used to. The first run of new Julia code can take longer while Julia compiles specialized machine code; later runs may be much faster. Its community is also smaller than Python's, so the available packages can influence our choice of tool. We will learn to measure performance, and we will use Python when its ecosystem suits the task.

Julia also supports LaTeX-style Unicode input in the read–evaluate–print loop (REPL) and VS Code. To write a scalar expression such as

$$v_1' \cdot (Av_2),$$

type a backslash command and press `Tab` after each symbol:

- `\alpha<Tab>` produces α.

- `\cdot<Tab>` produces ·.
- `v\_1<Tab>\prime<Tab> \cdot<Tab> (A * v\_2<Tab>)` produces $v_1' \cdot (A * v_2)$.

This produces the mathematical form shown above: variable names can carry the same Greek letters, subscripts, primes, and operators that appear in the calculation.

Reading the equations alongside the code

Try matching a line of code to the corresponding step in the model. This is often a useful way to spot a sign error, a misplaced index, or incompatible dimensions, and to check which assumptions the calculation uses.

0.4 Why Python?

Python is the dominant general-purpose language of the scientific data ecosystem. Its libraries are widely used for numerical arrays, plotting, optimization, astronomy, machine learning, symbolic algebra, image analysis, and experimental control. We will introduce Python after the core computational ideas are established, so that we can recognize the same scientific workflow in a second language.

Python is particularly valuable because it connects an analysis to the wider research environment. A laboratory may provide a Python script for an instrument, a collaborator may share a Jupyter notebook, or a public archive may publish an example using a specialized Python library. Learning Python lets us read and adapt those workflows, understand what each step contributes, and check the result. The language is also useful beyond numerical physics: file handling, web services, automation, databases, and data visualization use the same basic tools.

Much numerical Python code sends the heavy calculations to compiled libraries such as NumPy and SciPy. This allows a short Python expression to perform substantial numerical work efficiently. A loop written in ordinary Python and a calculation handled by a numerical library can therefore have quite different running times. When we examine performance, we will look at where the work happens and measure the time it takes.

Python is also readable enough that a short analysis often reads like an instruction to a colleague. The listing below is a supplied preview, not a lesson: we have not yet taught Python syntax, and this code is not assessed. Read it to see the shape of an analysis we can later revisit. Here `dropna` removes rows with missing entries in the named columns, `** 2` squares a column element by element, and `np.polyfit(..., 1)` fits a first-degree polynomial and returns its coefficients, so `fit[0]` is the slope:

```
clean = data.dropna(subset=["time_s", "position_m"])
clean["time_squared"] = clean["time_s"] ** 2
fit = np.polyfit(clean["time_squared"], clean["position_m"], 1)
gravity = 2 * fit[0]
```

The names are doing scientific work: the reader can see which columns were used, which transformation was made, and why the fitted slope is related to gravity.

With so many Python packages available, we will sometimes find several tools for the same task. We will choose well-documented ones, understand how they apply to our model, and record the package environment so that we can return to the analysis later. Adapting a library example becomes much easier when we can explain what each step does and check its result.

0.4.1 A complete example: the signal at both LIGO sites

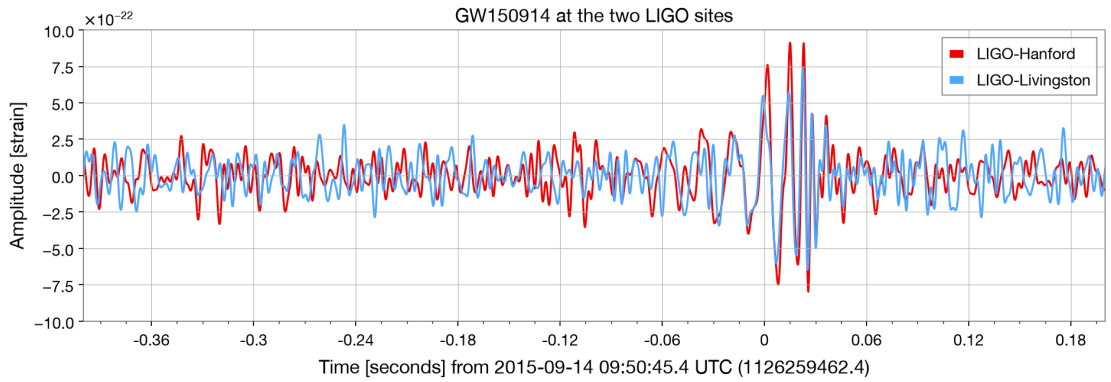
GW150914 was the first direct detection of gravitational waves and the first observed binary black-hole merger. We can compare the public Hanford and Livingston strain (i.e., how much space itself was distorted at two locations in the USA) around the event with the following Python example, adapted from the GWpy GW150914 filtering example. It fetches 32 seconds from each site, applies the same 50–250 Hz band-pass and 60, 120, and 180 Hz notch filters, and displays the central 0.6 seconds.

The code below is an "as is" preview: we have not yet learned the Python syntax or the GWpy library, and none of it is assessed this semester. Its here to get an idea what the workflow in Python looks like, and to demonstrate how extensive the Python ecosystem is. One can find packages for almost anything.

A list comprehension such as `[filter_design.notch(f, hdata.sample_rate) for f in (60, 120, 180)]` builds one filter entry per frequency; the star in `concatenate_zpks(bp, *notches)` unpacks that list into separate arguments; and `lfilt *= -1` multiplies the series in place by minus one.

The filters, the time shift, and the sign flip are the operations that make the two traces comparable.

```
from gwosc.datasets import event_gps
from gwpy.timeseries import TimeSeries
from gwpy.signal import filter_design
from gwpy.plot import Plot
t0 = event_gps("GW150914")
hdata = TimeSeries.fetch_open_data("H1", t0 - 16, t0 + 16)
ldata = TimeSeries.fetch_open_data("L1", t0 - 16, t0 + 16)
bp = filter_design.bandpass(50, 250, hdata.sample_rate)
notches = [filter_design.notch(f, hdata.sample_rate) for f in (60, 120,
    180)]
zpk = filter_design.concatenate_zpks(bp, *notches)
hfilt = hdata.filter(zpk, filtfilt=True)
lfilt = ldata.filter(zpk, filtfilt=True)
lfilt.shift("6.9ms")
lfilt *= -1
plot = Plot(figsize=(12, 4))
ax = plot.gca()
ax.plot(hfilt, label="LIGO-Hanford", color="gwpy:ligo-hanford")
ax.plot(lfilt, label="LIGO-Livingston", color="gwpy:ligo-livingston")
ax.set(xlim=(t0 - 0.4, t0 + 0.2), ylim=(-1e-21, 1e-21),
       title="GW150914 at the two LIGO sites", ylabel="Amplitude [strain
    ]")
ax.set_xscale("seconds", epoch=t0)
ax.legend()
plot;
```



This is the actual overlay returned by these commands. Each downloaded series has 131,072 samples at 4,096 Hz over 32 seconds; the wider interval keeps filter-edge transients far from the displayed event window. The Livingston series is shifted forward by 6.9 ms, the measured inter-site delay reported for this event, and multiplied by -1 to account for the detectors' relative orientations. These operations make the waveform shapes directly comparable; they are stated explicitly rather than fitted here. The trailing semicolon suppresses a duplicate notebook representation; in a standalone script we can call `plot.show()` or save the figure. The reproduction script and recorded versions are in `graphics/generate_gwpy_example.py` and `graphics/gw150914-gwpy-output.json`.

GW150914: what does this graph measure?

Two black holes merged and emitted gravitational waves. A passing wave changes the relative arm length in an interferometer: the strain $h = \Delta L/L$ is dimensionless. The detector output contains noise as well as any signal. The band-pass suppresses frequencies outside 50–250 Hz, the notch filters suppress narrow mains harmonics, and forward-backward filtering avoids an artificial phase delay. Filtering does not by itself establish a detection. Agreement between two independently located detectors after the documented delay and orientation correction is much stronger evidence than one trace alone, but a real analysis also quantifies noise significance and compares waveform models. GW150914 was observed on 14 September 2015 and announced in 2016. Both LIGO Hanford and Livingston observed the event; Virgo was not observing then. The 2017 Nobel Prize in Physics was awarded to Rainer Weiss, Barry C. Barish, and Kip S. Thorne for decisive contributions to the LIGO detector and the observation of gravitational waves. Calibration, noise, filtering, and waveform assumptions constrain the interpretation. The public GWOSC strain is distributed under CC BY 4.0; see `https://gwosc.org/GW150914/`.

0.5 Why Learn Both?

Once we have worked through a physical calculation in Julia, much of it will feel familiar when we encounter it in Python. The model and its units stay the same, even as the notation for storing data or calling a library changes. A case with a known answer gives us a starting check in either language; plots and residuals help us explore the result. Learning a second language gives us a chance to recognize these ideas in another setting.

We begin with Julia because its numerical notation lets us follow the structure of a physical calculation closely. Later, Python will open up the larger ecosystem we encounter in laboratories, research groups, and public scientific software. By the end of the course, we

should be comfortable reading code in both languages, choosing a tool for a particular task, and explaining the physics behind the calculation.

0.6 How the Year Is Organized

Semester A is primarily a Julia semester. We begin immediately with a supplied laboratory workflow for inspecting measurements, fitting a straight line, plotting residuals, and interpreting parameters with units; independent file and table work follows in Week 7. We then build the programming foundations and numerical methods needed to perform such analyses independently. In Week 12, the Tycho Brahe–Kepler investigation combines these skills in one reproducible analysis.

Semester B introduces Python by transferring methods we already understand, then extends them to eigenproblems, Monte Carlo simulation, nonlinear and uncertainty-aware fitting, Fourier analysis, partial differential equations, performance, machine learning, and symbolic workflows. Weekly assignments provide required practice and readiness checks; computerized assessments place greater weight on solving an unfamiliar problem independently and explaining the physical meaning of the result.

0.7 What We Will Learn

To turn a physics problem into a program, we first need to decide how the calculation will proceed. Which values do we start with? What changes at each step, and when should we stop? We will learn to express those choices through *algorithmic thinking*, choosing a representation for the data and breaking the calculation into steps we can follow and test. Simple cases and physical expectations will help us make sense of the output.

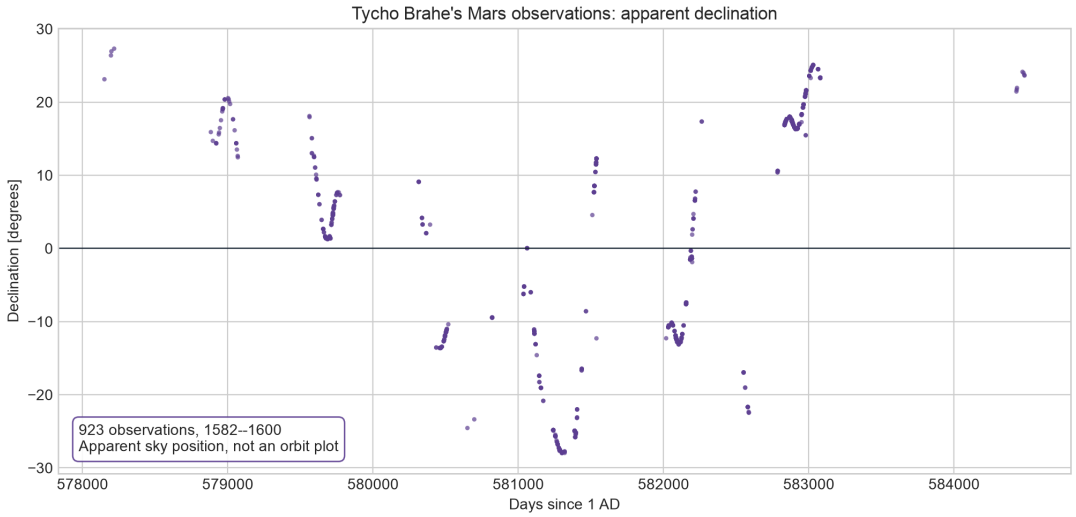
We will then use those habits to:

- read experimental and observational data from files;
- visualize data, models, residuals, and uncertainty;
- fit linear and nonlinear models and interpret their parameters;
- solve equations, integrate functions, and solve ordinary differential equations numerically;
- work with vectors, matrices, Fourier transforms, random processes, and simple PDEs;
- write readable, testable, reproducible scientific code in Julia and Python.

0.8 Teaser: Repeating Kepler's Mars Analysis

At the end of the first semester we will use observations associated with Tycho Brahe's Mars programme to reconstruct a version of Kepler's argument. This data set will become evidence for testing Kepler's first law: planetary orbits are ellipses with the Sun at one focus. We will take observations made at different dates, turn tabulated angular positions into a common geometric description, fit competing orbital models, and show quantitatively why an ellipse succeeds where a circle fails.

The plot below shows 923 measurements of Mars's apparent declination, its north–south angle in the sky, from 1582–1600. We can already see a pattern in how that angle changes over time. To connect it to the shape of the orbit, we will need to account for the viewing geometry and compare the predictions of circular and elliptical models. The declination graph on its own leaves that question open.



Tycho recorded dates and measured positions in tables. Turning those entries into a graph gives us a first view of the observations; building an orbital model will help us understand what produces the pattern. Along the way, we will draw on ideas from the courses you are taking alongside this one:

- **Mechanics:** central forces, orbital motion, and the physical interpretation of an ellipse with the Sun at a focus;
- **Mathematics:** vectors, coordinate transformations, conic sections, matrices, and least-squares fitting;
- **Laboratory:** uncertainty, residuals, data provenance, and the distinction between a good-looking curve and a supported conclusion.

By the time we reach this investigation, we will have the tools to write the analysis ourselves and use historical observations to examine the central empirical claim of Kepler's first law. For now, the graph gives us a question to carry through the semester. We will develop the geometry and fitting methods when we are ready to use them.

0.8.1 Obtaining the course repository

The student repository is `nshaviv/physprog`. We open a terminal in a parent directory of our choice and run:

```
git clone https://github.com/nshaviv/physprog.git
cd physprog
```

If Git is not installed, open the repository page and use *Code* → *Download ZIP*, then unpack the archive and open the folder; the remaining steps are identical.

Git creates the local course folder. We run the environment commands from this new repository root and open it in VS Code. The student README records which lectures, homework and solutions have been released. Detailed installation and update instructions are in `STUDENT_SETUP.md` at the repository root. The notebooks can be read from their saved outputs; opening them interactively in VS Code with the Jupyter extension or in JupyterLab is optional. Our personal working location need not match the instructor's computer.

0.9 The course environment: install once, run reproducibly

Install Julia 1.12 using `https://julialang.org/install/`; install Visual Studio Code and its official Julia extension using `https://www.julia-vscode.org/docs/stable/`

`gettingstarted/`. In a new terminal, confirm that the command found on our path is the intended Julia:

```
julia --version
```

```
julia version 1.12.6
```

Confirm that the output begins with `julia version 1.12`. If another version appears, restart the terminal after installation and check which Julia installation it finds.

Open the repository root in VS Code. This is the folder that contains `Project.toml` after cloning. Run the commands below from that root. The option `--project=.` activates the recorded course environment; the following `-e` option tells Julia to evaluate its argument.

```
julia --project=. -e 'using Pkg; Pkg.instantiate()'
```

Julia reads `Project.toml` and the recorded manifest, downloads any missing recorded packages, and reports package installation or precompilation. Run this once while connected to the network. Do not put `Pkg.add` or `Pkg.update` in course work: those commands would change an environment instead of using the recorded one.

Then run the course verification command with the same project selection:

```
julia --project=. verify_environment.jl
```

```
Course Julia environment verified.
```

The script checks the configured environment and ends with its success message. It may create temporary plot artifacts outside the repository. If it reports a missing package, close old Julia REPLs, make sure VS Code has selected the course environment, and repeat the instantiate command.

In VS Code, select this same course environment for the Julia extension. A file run from the editor sends values and `println` text to the integrated Julia REPL. The editor is where we write code; the REPL is the running Julia session that evaluates it. This distinction becomes useful in Chapter 1.

Optional: running the notebooks in VS Code

Every notebook already contains its saved outputs, so we can read it without running anything. To run one interactively:

1. In VS Code, install the *Jupyter* extension (the *Julia* extension stays for `.jl` files).
2. Open the repository root, then open a `.ipynb` file and use *Select Kernel* to choose `Julia 1.12` for the lecture and recitation notebooks, or the course Python environment for the Chapter 0 Python companions.
3. Run cells one at a time, or use *Restart Kernel and Run All* to reproduce a chapter from a clean start.

The same notebooks also open in JupyterLab. Running them is an option, not a requirement.



1. First Julia Calculations and a Guided Ohm-Law Fit

We begin a scientific calculation by predicting what should happen, making the calculation visible, and asking whether the result has the right units and meaning. Today we use Julia as a calculator and then follow a supplied data-to-fit-to-figure recipe. The recipe contains vectors and plotting commands that we will learn independently later.

1.1 Where do we see a result?

In the Julia REPL, typing an expression evaluates it and displays its value. In a VS Code file, evaluation can also show a value in a result display. That display is convenient, but it is not a durable part of a script's report. `println` explicitly sends text to the REPL, terminal, or script output; use it when our reader must see a result.

```
1 + 1
println(1 + 1)
```

```
2
```

Evaluating the first line in the REPL or an editor display shows `2`. The second line prints `2` to the REPL output. A script may evaluate many expressions without displaying them, so we use `println` for a deliberate textual report.

1.2 Julia as a calculator

Julia has familiar arithmetic, parentheses, powers, and mathematical functions. Most of the expressions below are *function calls*: a function name followed by parentheses, such as `exp(1)`. The call asks the named function to operate on the values in parentheses and returns a result. Arguments of `sin` and `exp` are in radians when they represent angles. `mod(a,b)` is the remainder after division by `b`; `div(a,b)` is integer quotient division, which discards the remainder for positive integers.

```
println(exp(1))
println(sin(pi / 2))
println(mod(17, 5))
println(div(17, 5))
```

```
2.718281828459045
1.0
2
3
```

Here `17 = 3*5 + 2`; the quotient is 3 and the remainder is 2.

We can enter mathematical Unicode symbols in the REPL and VS Code. Type `\pi<Tab>` to obtain π. In `2pi` or 2π, Julia interprets the numeric coefficient as multiplication by `pi`; it is not one variable name. Between two variable names we must write the multiplication sign explicitly, as in `mass * speed`.

```
println(2pi)
println((2 + 3) * 4)
println(2^3)
```

```
6.283185307179586
20
8
```

1.3 Names for values before physics

An assignment such as `x = 2` attaches the name on the left to the value calculated on the right. It is an instruction, not the algebraic statement $x = 2$ for all time. We can then use names to expose a calculation.

```
x = 2
y = 3
println(x + y)
println(x^2 + y^2)
```

```
5
13
```

We keep names free of unit suffixes and record each unit in a short trailing comment where the value is defined, for example `voltage = 2.0 # V`. Julia neither infers nor enforces units from the comment, so the comment and any function documentation carry them.

1.4 A guided measurement-to-model workflow

The following recipe restores the full explanation of our first analysis. We supply its file-reading, vector, and plotting steps. We are expected to run and interpret these steps now; independent CSV/DataFrames work belongs to Week 7, and fitting mastery to Week 8. The mathematical derivation is optional background, separate from the graded one-third-point bonus. We do not define functions until Week 5.

Open the supplied Week 1 notebook or run the saved program from the directory containing `ohm_law.csv`. The shared course environment already declares CSV, DataFrames, and Plots; installation is described in Chapter 0.

What is a CSV file?

A *comma-separated values* (CSV) file stores a table as plain text. The first line usually gives the column names; each later line is one row, and commas separate the entries.

For example,

```
current_A,voltage_V
0.10,0.52
0.20,1.01
```

is a two-column table with current in amperes and voltage in volts. Spreadsheet and laboratory programs can usually export a table as CSV. Plain text is useful because the file can be inspected by a person and read by many programs. We will use the `CSV.jl` package to read the file and `DataFrames.jl` to keep the table organized.

An ohmic conductor

For an ohmic conductor in an approximately fixed state and weak-field/linear-response regime, $V = RI$. We take applied voltage V in V and positive current I in A in the chosen positive direction; $R = V/I$ has units Ω, and conductance $G = 1/R$ has units S. Scattering establishes steady drift and helps set resistance, but rapid collisions alone do not guarantee linearity. Heating, contacts, strong fields, and non-ohmic devices can invalidate the model. Its ideal check is $I = 0 \Rightarrow V = 0$.

Math background: a straight-line model

The model $y = ax + b$ has a slope a and an intercept b. The slope tells us how much y changes when x increases by one unit; the intercept is the model value at $x = 0$.

How we introduce a new programming idea

Before using a new construct, we will ask why it is needed, define what it is, show its general Julia form, and try the smallest useful example. We will then use it in a richer calculation and finally connect it to a scientific task. When reading code, follow the same order: purpose, form, simple case, and application.

Ways to run Julia in Week 1

- **A system terminal:** open Terminal, PowerShell, or a Linux shell and type `julia`. This starts a Julia REPL, where `1+1` is evaluated immediately.
- **The VS Code terminal:** open **Terminal** > **New Terminal** inside VS Code and type `julia`. It is the same REPL, but beside the course files.
- **The VS Code Julia REPL:** use the Command Palette and choose *Julia: Start REPL*. With a `.jl` file open, `Shift+Enter` sends the current line or selected code to that REPL and starts it if needed.
- **A complete saved program:** use **Run** > **Run Without Debugging** to execute the active `.jl` file from top to bottom. This is the normal choice for submitted work because the analysis is saved and repeatable.

Later we will introduce notebooks, whose cells are more interactive. Do not confuse a notebook cell with a saved analysis: both still need clear inputs, code, plots, and conclusions.

1.4.1 Build the analysis one visible step at a time

Step 1: load the tools. The first line of a script says which installed packages the program will use.

```
using CSV, DataFrames, Plots
```

The keyword `using` makes the public tools from these packages available in this file. Here, `CSV` reads the text file, `DataFrames` supplies a table structure, and `Plots` creates figures. This is not a calculation yet: it is the program stating which tools it needs.

Step 2: load and inspect the table. The next lines open the file and show its first five rows.

We need two basic code patterns before doing that. A *function call*, which we saw with `exp` and `sin`, asks a named tool to perform an operation. Its general form is `function_name(input1, input2)`; for example, `println("ready")` calls `println` with one text input. An *assignment* gives a result a reusable name. Its general form is `name = expression`; Julia evaluates the right side first and then stores the result under the name on the left. The smallest example is `x = 2`. The next block combines these patterns with a real data file.

The second input to `CSV.read`, `DataFrame`, names the table type we want back: a table with named columns. The helper `first(data, 5)` returns the first five rows of that table so we can inspect them.

```
root = get(ENV, "PHYSPROG_HOME", pwd())    # the course folder
candidates = [joinpath(root, "assets", "data", "ohm_law.csv"),
              joinpath(root, "reveal", "week01", "ohm_law.csv")]
datafile = candidates[findfirst(isfile, candidates)]
data = CSV.read(datafile, DataFrame)
println(first(data, 5))
```

With the supplied file, the five printed data rows are:

Row	current_A	voltage_V
1	0.1	0.52
2	0.2	1.01
3	0.3	1.52
4	0.4	2.0
5	0.5	2.49

Read `CSV.read(datafile, DataFrame)` from left to right. `CSV.read` means "use the function `read` supplied by the `CSV` package." Its first input is the file name; its second input asks for a `DataFrame`, a table with named columns. The equal sign stores the resulting table under the name `data`. The next line takes the first five rows and prints them in the REPL or terminal. This is a first scientific check: are the column names right, are the numbers plausible, and did we open the intended file?

Step 3: name the physical quantities. Extract the two columns into unit-free variable names whose definitions record the units in a short comment.

A `DataFrame` gives each column a name. The access form `table.column_name` selects one complete column; for example, `data.current_A` selects the measurements stored under the heading `current_A`. We assign the two selected columns meaningful names:

```
current = data.current_A   # A
voltage = data.voltage_V   # V
```

The names `current` and `voltage` are *variables*: labels that let later instructions refer to values stored by the program. The equal sign assigns the value on its right to the variable on its left; it does not state a permanent mathematical identity. The dot in `data.current_A` selects the column named `current_A`. We keep the identifiers unit-free and read the unit from the trailing comment, which Julia ignores. Document units where values are defined, in function docstrings or interface documentation, and at call sites when the units would otherwise be ambiguous. We use integer counts and decimal measurement values here without developing Julia's type system; Week 2 will name and inspect these simple types explicitly.

Step 4: make the data visible. Before fitting anything, plot the measurements.

A *vector* is an ordered list written inside square brackets, such as `[0, 1]`. A *scatter plot* represents every pair of values from two vectors by one point, so it lets us see trends and unusual measurements before trusting a model. The basic call has the form `scatter(x_values, y_values)`. For example, `scatter([0, 1], [0, 1])` draws two points. Optional named settings have the form `keyword=value`; they add labels or change appearance without changing the measured data. For instance, `markerstrokewidth=0` removes the outline drawn around each marker so overlapping points stay readable. We now apply that format to current and voltage:

```
scatter(current, voltage,
    xlabel="current I [A]", ylabel="voltage V [V]",
    label="measurements", markerstrokewidth=0)
```

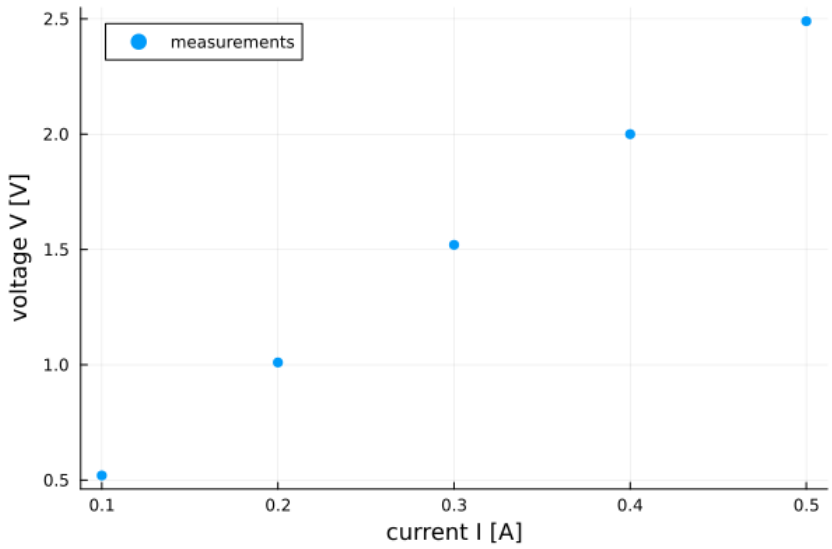


Figure 1.1: Graphical output of the scatter call: the measured voltage rises approximately linearly with current.

The first two inputs to `scatter` are the horizontal and vertical coordinates. The later inputs are *keyword arguments*: each has a name, such as `xlabel`, followed by an equal sign and a value. They control the appearance of the figure. Labels and a legend are not decoration; they tell a reader what was measured and in which units.

The plot is part of the calculation

Never trust a fit that you have not plotted. A numerical answer can hide an outlier, a nonlinear trend, a unit mistake, or swapped columns.

Step 5: fit a line from its definition. For a trial line $V = RI + V_0$, the residual for measurement i is the vertical mismatch

$$r_i = V_i - (RI_i + V_0).$$

We choose the line that minimizes the sum of squared mismatches,

$$S(R, V_0) = \sum_{i=1}^{n} [V_i - (RI_i + V_0)]^2 .$$

Squaring prevents positive and negative mismatches from cancelling and penalizes a large mismatch more than a small one. This is the unweighted linear least-squares definition: it treats all measurements as equally reliable. We will study minimization later in the course. For Week 1, use the resulting direct formulas rather than a general optimizer or a matrix method.

For a smallest exact example, the three points $(0, 1)$, $(1, 3)$, and $(2, 5)$ all lie on $y = 2x + 1$. Their best-fit slope is 2, their intercept is 1, and every residual is zero. Real measurements do not lie exactly on one line, so the Ohm's-law calculation below is the richer case.

The direct linear-fit formulas

Write $\overline{I}$ for the average current and $\overline{V}$ for the average voltage. The best-fit slope and intercept are

$$R = \frac{\sum_{i=1}^{n}(I_i - \overline{I})(V_i - \overline{V})}{\sum_{i=1}^{n}(I_i - \overline{I})^2}, \qquad V_0 = \overline{V} - R\overline{I}.$$

The numerator is positive when current and voltage tend to increase together. The denominator measures how much the current values spread around their average. A fit is impossible if all current values are identical, because then no slope can be determined. The intercept is chosen so that the fitted line passes through $(\overline{I}, \overline{V})$.

Optional mathematical background: where do the fitting formulas come from?

This box is for those of us who want to look ahead. It is not required for Week 1. We are looking for a minimum of $S(R, V_0)$, which depends on two adjustable parameters. At a smooth minimum, changing either parameter a tiny amount cannot decrease S, so both partial derivatives must vanish.

First differentiate with respect to the intercept:

$$0 = \frac{\partial S}{\partial V_0} = -2 \sum_{i=1}^{n} [V_i - (RI_i + V_0)] .$$

Dividing by $-2n$ and rearranging gives $V_0 = \overline{V} - R\overline{I}$. The fitted line therefore passes through the average data point.

Next differentiate with respect to the slope R:

$$0 = \frac{\partial S}{\partial R} = -2 \sum_{i=1}^{n} I_i [V_i - (RI_i + V_0)] .$$

Substitute the expression for V_0. The residual becomes $(V_i - \overline{V}) - R(I_i - \overline{I})$. Using $\sum_i (I_i - \overline{I}) = 0$ and $\sum_i (V_i - \overline{V}) = 0$, the remaining equation is

$$\sum_{i=1}^{n} (I_i - \overline{I})(V_i - \overline{V}) = R \sum_{i=1}^{n} (I_i - \overline{I})^2.$$

Solving this equation for R gives the direct slope formula above; substituting that slope back into $V_0 = \overline{V} - R\overline{I}$ gives the intercept. In Week 8, least-squares mastery and later minimization will make this reasoning a regular course tool.

The formulas must apply the same operation to every measurement vector. A dot on an arithmetic operator tells Julia to apply that operation element by element. The general pattern is `vector .operator value`. The symbol `#` begins a comment: Julia ignores the rest of that line, while a reader can use it as an explanation. For example,

```
values = [1, 2, 3]
squared = values .^ 2     # [1, 4, 9]
```

We now apply dotted operations to all current and voltage measurements. Two vector summaries come first: `length(current)` returns the number of measurements, and `sum(current)` adds the entries of the vector. We use both to compute the averages. The code implements the formulas directly, so every quantity remains visible:

```
n = length(current)
I_bar = sum(current) / n   # A
V_bar = sum(voltage) / n   # V
dI = current .- I_bar      # A
dV = voltage .- V_bar      # V
resistance = sum(dI .* dV) / sum(dI .^ 2)   # ohm
offset = V_bar - resistance * I_bar   # V
```

`length(current)` counts the measurements and `sum(...)` adds the entries of a vector. The dot in `.-`, `.*`, and `.^` means "apply this operation to matching elements." Thus `dI .* dV` contains one product $(I_i - \overline{I})(V_i - \overline{V})$ for each measurement, ready for `sum` to add. Julia deliberately distinguishes element-by-element operations from matrix operations; ordinary matrix multiplication is ill-defined for two vectors of matching measurements. In Week 1, use the dotted forms whenever the calculation is meant to act on every measurement separately.

Step 6: put the model on the same graph and report the result.

We build the model figure from a fresh copy of the measured points, then add the fitted line to that same figure. In the form `plot!(x, y)`, the exclamation mark indicates that an existing plot is modified; `savefig("name.png")` then saves the current plot. Calling `plot()` with no data starts a new, empty plot. To report numbers, `$(expression)` inserts a calculated value into a string, and `round(value, digits=3)` limits its displayed decimal places. A tiny example of each form, on its own plot, is:

```
plot()
plot!([0, 1], [1, 0])
savefig("example.png")
rounded = round(1.2345, digits=2)
println("value = $(rounded)")
```

```
value = 1.23
```

The full scientific reporting block re-creates the measurement scatter and adds the fitted model, so the toy line cannot remain in the figure. On the model line, `linewidth=2` sets the line's width:

```
scatter(current, voltage,
    xlabel="current I [A]", ylabel="voltage V [V]",
    label="measurements", markerstrokewidth=0)
plot!(current, resistance .* current .+ offset,
    label="least-squares model", linewidth=2)
savefig("ohm_law_fit.png")

println("R = $(round(resistance, digits=3)) ohm")
println("offset = $(round(offset, digits=3)) V")
```

```
R = 4.93 ohm
offset = 0.029 V
```

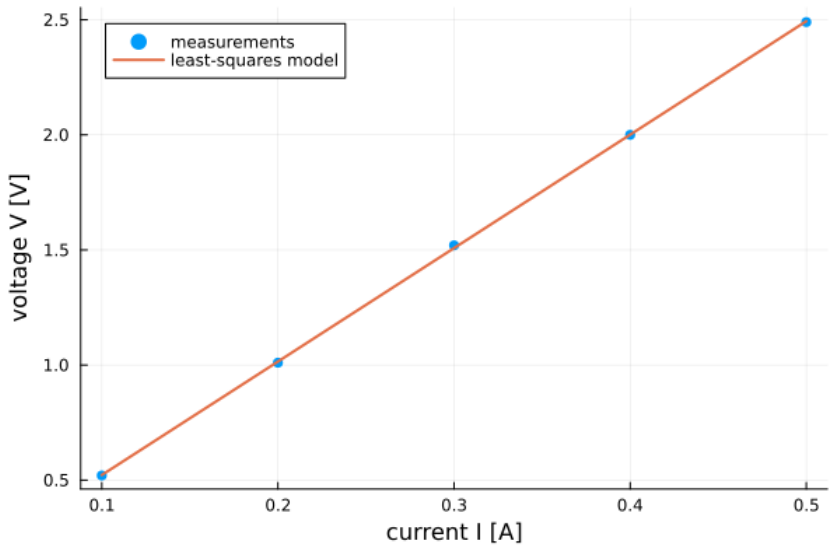


Figure 1.2: Graphical output after `plot!`: the least-squares model is drawn on the measurements.

The dot in `resistance .* current` multiplies the scalar resistance by every current value. The dot in `.+ offset` then adds the intercept to every result. The exclamation mark in `plot!` means "add to the existing plot" rather than create a new one. The following `savefig` call saves the current plot as a PNG file in the working folder.

Step 7: inspect the residuals. A fitted line is a model, not proof that the model is good. Plot the residuals to check whether they fluctuate around zero without an obvious pattern. The mutating form `hline!` adds a horizontal line to the current plot, which we use for the zero reference:

```
residual = voltage .- (resistance .* current .+ offset)   # V
scatter(current, residual,
    xlabel="current I [A]", ylabel="residual V - model [V]",
    label="residuals", markerstrokewidth=0)
hline!([0], label="zero residual")
savefig("ohm_law_residuals.png")
```

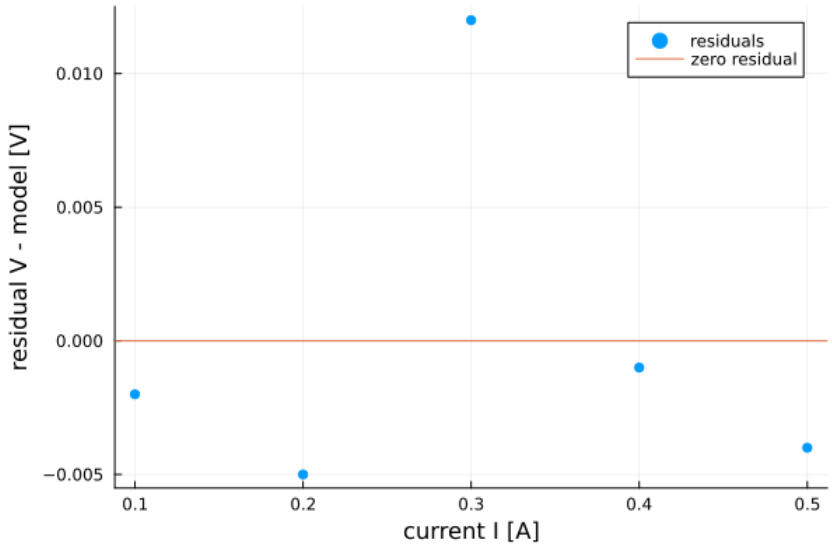


Figure 1.3: Graphical output of the residual calculation: all residuals are small and lie on both sides of zero.

The second `savefig` call saves the residual plot that is current at that point. If the residuals curve systematically above and below zero, a straight line may be the wrong model. If one residual is much larger than the others, inspect that measurement rather than silently trusting the fit. At this point we have a complete, small scientific program: it reads data, checks it, saves figures, computes model parameters, and communicates their meaning with units.

Measurement scatter, model error, and reported digits

Measurement scatter is variation in repeated observations caused by instrument resolution and experimental conditions. Model error is a systematic mismatch between the chosen model and the physical system; a curved residual pattern is evidence for such a mismatch. These are different from display rounding. Printing three decimal places makes output consistent, but it does not prove that all three decimals are physically significant. The justified number of significant figures must eventually follow from measurement uncertainty, which we will treat more fully later.

Guided practice for this workflow is in Recitation Chapter 1, *Free-Fall Data*.

Physics background: units in fitted parameters

If y has units of volts and x has units of amperes, the slope has units of volts per ampere, namely ohms. The intercept has the same units as y. Unit analysis is a simple way to catch fitting mistakes. Later, a package such as `Unitful.jl` can carry units together with values in the code; the physical reasoning remains the same.

1.5 Recitation: measuring a spring constant

We transfer the same analysis to a spring. Our measured pairs are extension x in metres and applied force F in newtons; the fitted model is $F = kx + F_0$. Unlike a fit with extension on the vertical axis, the slope here is directly the spring constant k in N/m. We first predict a positive slope and check that the intercept has units of force.

Before the session, we open the Week 1 recitation notebook and locate its supplied data and fitting cells. During the first 20 minutes we inspect the axes, follow one fit, and explain the slope, offset, and one residual. We then begin the separate free-fall homework with staff support. A useful checkpoint is to explain why reversing the axes changes the units and the meaning of the fitted slope; in noisy data, refitting the reversed axes is not generally the same as taking the reciprocal.

1.6 Core homework: infer gravity from free-fall measurements

We investigate whether measured fall distances agree with release from rest under approximately uniform gravity. We take distance positive downward, so $y(t) = \frac{1}{2}gt^2 + b$, with $g > 0$ in m/s^2 and offset b in m. The supplied transformation `t_s .^ 2` makes squared time the horizontal coordinate. We fit $y = at^2 + b$ and interpret $g = 2a$; the factor of two comes from the motion equation, not from a software convention.

We work in these stages:

1. Inspect the supplied `free_fall.csv`: identify the time and distance columns, their units, and the number of observations.
2. Predict the shape of y versus t and of y versus t^2 before running the fit. Apply the supplied recipe and retain the predictions and measured-minus-predicted residuals.
3. Make one figure: the observed points as distinct markers with the fitted model overlaid, and every axis labelled with its quantity and units.
4. Save the figure as PNG or PDF and report the fitted g and offset. Explain whether the residuals support the model over the measured interval and name one assumption that the fit cannot establish.
5. Submit your script and the figure on the course Moodle page, as specified on the assignment sheet.

We submit a complete script together with the figure; defining a Julia function is not part of Week 1. Week 1 is graded by hand on Moodle, pass/fail. It is worth keeping the assignment sheet close by, since that is where the required columns, the figure, and the quantities to report are listed. Rounding a displayed answer must not round the values used for predictions.

1.7 Optional bonus: make a scientific figure communicate

This bonus is optional, is not a prerequisite for later work, and is not examined. A completed bonus is worth **one-third of a final-grade point**.

After 10–15 minutes of self-contained reading, we improve a supplied minimal Ohm-law figure. Here voltage is the horizontal coordinate and current the vertical coordinate, so the model slope is conductance in S, not resistance in Ω. We identify the measured-current series and fitted-current series before changing their appearance.

We add the prescribed title, labels with units, legend entries, observation markers, model line, frame, and canvas size, then export PNG and PDF. The numeric fragment supplies the validated aligned series; the images are assessed separately against the explicit checklist. We check that changing a label or line style does not alter a measurement. The bonus sheet specifies the exact strings and filenames so that completeness, rather than taste or pixel matching, determines success.

1.8 Going further: Lissajous curves

In this optional, unexamined investigation we use a supplied plotting recipe for

$$x(t) = A\cos(at + \delta), \qquad y(t) = B\sin(bt).$$

Here t is in seconds, a, b are angular frequencies in rad/s, δ is a phase in radians, and A, B set the coordinate amplitudes. These curves describe two independent sinusoidal motions viewed together; they are not trajectories under a central gravitational force.

We keep amplitudes fixed and compare frequency ratios $1:1$, $2:1$, and $3:2$. Then, for one ratio, we vary the phase from 0 to $\pi/2$. We save two contrasting plots and write a conjecture about closure and symmetry. We check whether a longer time interval changes an apparently open curve: a finite plotting window is not evidence of nonclosure. This activity adds no required programming syntax and is separate from the graded bonus.



2. Week 2: Values, Decisions, and a Block on an Incline

2.1 Lecture: From Variables to an Algorithm

In Week 1 we gave names to measurements and calculated new values from them. Before adding decisions, we will slow down and make that basic idea explicit: what a variable is, what assignment does, and how values change as Julia executes one line after another.

2.1.1 A variable gives a value a name

A variable is a name that refers to a value. A useful name lets us discuss the meaning of a number instead of repeatedly writing the number itself:

Assignment has the general form `name = expression`. Julia calculates the expression on the right and makes the name on the left refer to the result. The smallest example is `x = 2`. A richer scientific calculation uses several meaningful names:

```
mass = 2.0          # kg
speed = 3.0         # m/s
momentum = mass * speed   # kg m/s

println(momentum)   # 6.0
```

```
6.0
```

The single equal sign is the *assignment operator*. Julia first evaluates the expression on the right of `=`; it then makes the name on the left refer to the resulting value. Thus the third line first multiplies `mass` by `speed`, then assigns the result to `momentum`.

Momentum of a moving mass

An object's momentum measures how much motion it carries: a heavier object and a faster object each carry more. For a point mass moving well below the speed of light, $\mathbf{p} = m\mathbf{v}$, with magnitude $p = mv$ along the direction of motion. Mass m is in kilograms, speed v in metres per second, and momentum in kilogram-metres per second $(\mathrm{kg\,m/s})$.

We can inspect a variable simply by writing its name in a notebook cell, or print it as part of a message:

```
momentum
```

```
println("momentum = $(momentum) kg m/s")
```

In a notebook the first line displays `6.0`; the printed report is:

```
momentum = 6.0 kg m/s
```

2.1.2 Julia follows the program in order

An assignment performs a calculation at that moment. It does not create a permanent algebraic relationship between variables:

```
x = 4
y = x + 1       # y is now 5
x = 10          # x changes, but y is still 5
```

When Julia executes `y = x + 1`, it uses the current value of `x`. Changing `x` later does not automatically recalculate `y`; we would need to execute `y = x + 1` again. This line-by-line order is especially important in notebooks, where cells can be run more than once or out of order.

2.1.3 A variable can be assigned a new value

The same name can refer to a new value later in the program:

```
temperature = 20.0   # C
temperature = temperature + 0.5

println(temperature)     # 20.5
```

```
20.5
```

The second line is not an algebraic equation. Julia reads the old value on the right, adds `0.5`, and assigns the new result back to the same name. Loops will use this pattern to advance time, increase a counter, or update a simulated position.

Names should communicate meaning

Names: Variable names are case-sensitive, cannot contain spaces, and should be used consistently. You can use unicode, and in particular greek symbols (e.g., type `\phi<Tab>` to get ϕ). Prefer meaningful names to unexplained names such as `a`, `b`, and `c`. **Units**: Julia generally stores just ordinary numbers without their units. However in physics, it is advisable to keep track of them. You can add unit suffixes to the names, e.g., `m_kg`, or alternatively, as we do here, record the unit in a short trailing comment where the value is defined, for example `time = 0.35 # s`, `height = 1.20 # m`, and `g = 9.81 # m/s^2`; This is not compulsory but very useful to avoid unit errors. In bonus assignment 2 (i.e., not part of the official course material), we will see that we can use a package `Unitful` that adds units to variables, allowing us to keep track them. Units should likewise be stated in function docstrings or interface documentation, and clarified at call sites when otherwise ambiguous.

2.1.4 Values have types

Variables can name different kinds of values. We already used an integer count, decimal measurements, and text labels in Week 1. This week we add Boolean comparisons and make four distinctions explicit because a program that reads input, chooses a branch, and decides when to stop must know what kind of value it is handling.

A *type* is a category of values that determines how Julia stores them and which operations make sense. For example, `5`, `0.35`, `"height"`, and `true` have different formats and represent an integer, a floating-point number, text, and a true/false value.

2.1.5 Four simple value types

The four types we need now

Julia type	Example	Typical scientific role
`Int`	`n = 5`	counts and loop indices
`Float64`	`t = 0.35 # s`	measurements and calculated quantities
`String`	`label = "height [m]"`	input text, labels, and messages
`Bool`	`ready = true`	true/false answers to conditions

The function `typeof(value)` lets us inspect a value's type. We use it for diagnosis, not as a replacement for understanding what the value means.

On the usual 64-bit course computers, `typeof(5)` prints `Int64`. Julia's name `Int` means the standard integer size for the computer and is an alias for `Int64` on these machines.

These types connect directly to Week 1:

```
current = [0.10, 0.20, 0.30, 0.40, 0.50]   # A
n = length(current)                  # Int
I_bar = sum(current) / n             # Float64, A
axis_label = "current I [A]"    # String
analysis_ready = true           # Bool

println(typeof(n))
println(typeof(I_bar))
println(typeof(axis_label))
println(typeof(analysis_ready))
```

An integer and a floating-point number can participate in one calculation. Julia converts the integer as needed:

```
2 * 0.35       # 0.7, a Float64
5 / 2          # 2.5, a Float64
Float64(5)     # 5.0
```

For now, these practical rules are sufficient. We do not need type annotations, abstract types, or the details of floating-point storage.

A type is not a physical unit

In `t = 0.35 # s`, the Julia type is `Float64`; the physical unit is seconds. The unit lives in the comment and in the relevant interface documentation, so a different `Float64` could represent metres or kilograms. Type checking and dimensional analysis answer different questions, and we need both.

2.1.6 Comparisons produce Boolean values

Programs need yes/no questions in order to validate data and choose what to do next. A comparison has the form `left_value operator right_value` and produces a `Bool`: either `true` or `false`. Start with direct questions whose answers are obvious:

```
2 < 3          # true
2 == 3         # false
5 != 4         # true
```

The main comparison operators are `<`, `<=`, `>`, `>=`, `==` (equal), and `!=` (not equal). The single equal sign `=` assigns a value; the double equal sign `==` asks a question.

We can combine Boolean values when one comparison is not enough. The operators `&&`, `||`, and `!` mean and, or, and not:

```
true && false      # false: both must be true
true || false      # true: at least one is true
!true              # false
```

The same formats can express a richer physical validity rule:

```
t = 0.35   # s
z0 = 1.20   # m
nonnegative_time = t >= 0
valid_height = z0 >= 0
valid_input = nonnegative_time && valid_height
```

2.1.7 A first decision with if

Calculating a Boolean answer is useful only if the program can act on it. An `if` statement is a conditional: it runs one block when its condition is `true`. An optional `else` block runs when the condition is `false`. The general format is:

```
if condition
    # commands when the condition is true
else
    # commands when the condition is false
end
```

Every conditional ends with `end`. If nothing should happen in the false case, we omit the `else` block. Here is the smallest two-way example:

```
temperature = 25.0   # C
if temperature > 30
    println("hot")
else
    println("not hot")
end
```

```
not hot
```

Sometimes continuing would make a calculation meaningless. Julia provides a deliberate stop with this format:

```
throw(ArgumentError("message"))
```

It stops execution and reports why. We will use it after a failed conversion and after physically invalid input.

2.2 Piecewise Kinematics as a Decision

Signed acceleration versus positive gravity magnitude

Height is positive upward, so gravitational acceleration is

$$g = -9.81 \text{ m/s}^2.$$

For initial height z_0 and velocity v_0,

$$z_{\text{free}}(t) = z_0 + v_0 t + \frac{1}{2} g t^2.$$

The three terms have units of metres. This dimensional check is independent of whether their code values are `Float64`.

Our no-bounce model does not allow the ball to continue below the ground:

$$z(t) = \begin{cases} z_{\text{free}}(t), & z_{\text{free}}(t) > 0, \\ 0, & z_{\text{free}}(t) \le 0. \end{cases}$$

The piecewise definition translates directly to a conditional:

```
t = 0.35   # s
z0 = 1.20   # m
v0 = 0.0    # m/s
g = -9.81   # m/s^2

if t < 0 || z0 < 0
    throw(ArgumentError("time and initial height must be non-negative"))
end

z_free = z0 + v0 * t + 0.5 * g * t^2   # m

if z_free > 0
    z = z_free
    state = "in air"
else
    z = 0.0
    state = "on the ground"
end

println("at t = $(t) s: z = $(round(z, digits=3)) m ($(state))")
```

```
at t = 0.35 s: z = 0.599 m (in air)
```

At $t = 0.35$ s this program reports approximately $z = 0.599$ m. The result passes three checks: time is in the model domain, the height is below its initial value, and every term in the position equation has units of metres.

2.2.1 Several branches

Use `elseif` when more than two cases are mutually exclusive and should be tested in order. The general format is `if`, followed by any number of `elseif` branches, an optional `else`, and one `end`. Only the first true branch runs. A small sign-classification example shows the structure:

```
value = -2
if value < 0
    description = "negative"
elseif value == 0
    description = "zero"
else
    description = "positive"
end
```

The same structure gives a more useful height classification:

```
if z == 0
    state = "on the ground"
elseif z < 0.20
    state = "close to the ground"
else
    state = "in air"
end
```

Nested decisions are sometimes necessary, but a flat `if`/`elseif`/`else` chain is easier to inspect when it expresses the same logic.

Three error categories

- **Syntax error:** Julia cannot parse the program, for example because an `end` or closing parenthesis is missing.
- **Runtime error:** the program starts and then reaches an invalid operation, for example raising a `String` to a power.
- **Logical error:** the program runs but implements the wrong algorithm or physics, for example using positive g while height is positive upward, or failing to advance time in a `while` loop.

Read the exact error and line number, isolate the smallest failing block, inspect values and types, check dimensions and signs, and test cases whose answers are already known.

We now make programs choose among physically distinct cases. Before a calculation can be trusted, we must know what kind of value we hold, convert deliberately, and state the domain in which a model applies.

2.3 Values have types

Julia values carry types. Integers, floating-point numbers, text, and truth values serve different purposes.

```
println(3, "  ->  ", typeof(3))
println(3.0, "  ->  ", typeof(3.0))
println("3", "  ->  ", typeof("3"))
println(3 < 4, "  ->  ", typeof(3 < 4))
```

```
3  ->  Int64
3.0  ->  Float64
3  ->  String
true  ->  Bool
```

The exact integer type can depend on the computer; the important distinction is `Int`, floating-point, `String`, and `Bool`.

An integer and a floating-point number can mix in ordinary arithmetic; Julia promotes the result where needed. Text is not silently a number. We choose conversions explicitly in both directions.

```
println(2 + 0.5)
text = "12.5"
mass = parse(Float64, text)   # kg
println(mass + 0.5)
println(string(mass, " kg"))
println(Int(7.0))
```

```
2.5
13.0
12.5 kg
7
```

`parse(Float64, "twelve")` throws an error because the text is not a floating-point numeral. `Int(7.2)` also throws an error: converting to `Int` is allowed only when the value is exactly integral. This protects us from quietly discarding information.

2.4 Booleans and gradual decisions

Comparisons produce Boolean values. We first ask one question, then choose between two cases, then add a third case with `elseif`.

```
temperature = -2.0   # C
println(temperature < 0)
if temperature < 0
    println("below freezing")
else
    println("not below freezing")
end
```

```
true
below freezing
```

```
if temperature < 0
    state = "ice"
elseif temperature == 0
    state = "melting-point"
else
    state = "liquid"
end
println(state)
```

```
ice
```

`ice`. Exactly one arm of an `if`/`elseif`/`else` chain runs. `=` assigns; `==` compares.

We combine conditions with `&&` (both), `||` (at least one), and `!` (not). A validity check often asks whether a value is a finite number. `isfinite(value)` returns `true` for ordinary finite numbers and `false` for `NaN`, `Inf`, or `-Inf`, which are not usable measurements. We can combine that check with a physical condition:

```
is_finite_positive = isfinite(2.0) && 2.0 > 0
println(is_finite_positive)
println(!is_finite_positive || 2.0 == 2)
```

```
true
true
```

2.5 Angles, logic, and validation before a calculation

A condition is a precise mathematical question

`a && b` is true only when both conditions are true; `a || b` is true when at least one is true. `xor(a,b)` is true when exactly one is true. These rules are useful for choosing mutually exclusive cases, but a model must still specify what happens at equality. A chained inequality such as $0 \le \theta < 90^\circ$ means both bounds must hold; we can spell it out with two comparisons joined by &&.

An angle needs an explicit unit. `sin` and `cos` expect radians, while `sind` and `cosd` expect degrees; `rad2deg` converts an angle from radians to degrees. The two-argument form `atan(y, x)` returns the quadrant-aware angle of the point (x, y): it uses both signs, not only their ratio, so it can distinguish opposite directions. A tiny example of each is:

```
println(true && false)
println(true || false)
println(xor(true, true))
println(xor(true, false))
println(sin(pi / 6))
println(sind(30.0))
println(round(rad2deg(atan(1.0, -1.0)); digits=1))
```

```
false
true
false
true
0.49999999999999994
0.5
135.0
```

The point $(-1, 1)$ lies at 135°, not -45°; a one-argument `atan(y/x)` could not tell those apart. At the origin there is no physical direction; an application must state a convention. Tiny differences between equivalent floating-point calculations are not different physics.

Text conversion and physical validation are separate

`readline()` returns text. `parse(Float64, text)` turns valid numerical text into a number; malformed text raises an error. Successful conversion is not enough: `NaN` and `Inf` are not finite measurements, and a finite negative mass is outside our model. We validate before a square root, division, or trigonometric construction. `throw(ArgumentError("message"))` stops the calculation; merely printing an error message does not.

Before testing a physical domain, we check that the value is finite: `isfinite(value)` is `true` only for ordinary finite numbers and `false` for `NaN` or `Inf`. The following fragment converts text and validates the result as a mass:

```
text = "2.5"  # reproducible example of text supplied by readline()
mass = parse(Float64, text)   # kg
if !isfinite(mass) || mass <= 0
    throw(ArgumentError("mass must be finite and positive"))
end
println("mass = ", mass, " kg")
```

```
mass = 2.5 kg
```

The earlier recovery-style `tryparse`/`nothing` examples are preserved for optional input-handling reading. The core route this week uses `parse` and explicit physical domain checks; neither route requires defining a function.

2.6 A piecewise friction model on an incline

Forces, signs, and assumptions

Take $+x$ uphill along a fixed incline and $+y$ normal to it. For a block of mass m (kg), angle θ (radians in `sin`/`cos`), and positive gravity magnitude g (m/s^2), gravity has downhill component $mg\sin\theta$ and normal force $N = mg\cos\theta$. Static friction can oppose the impending downhill motion up to $f_{s,\max} = \mu_s N$. If it cannot hold, kinetic friction $f_k = \mu_k N$ acts uphill and $a = (-mg\sin\theta + \mu_k N)/m$. We assume Coulomb friction, rigid contact, no drag, and $0 \le \theta < 90°$. The model does not describe rolling, changing contact, or velocity-dependent friction. At $\theta = 0$, the required static friction is zero; if $\mu_k = 0$, sliding gives $a = -g\sin\theta$.

We validate before using an input. `throw(ArgumentError(...))` immediately stops the calculation and tells its caller that a documented input condition failed. It is different from `println`, which merely reports text and continues. The membership test `value isa T` asks whether a value belongs to type `T`; `Real` names Julia's family of real-number types, which includes ordinary integers and floating-point values. `Bool` needs special care here: because `true isa Real` is also `true`, a check for `Real` alone would accept `true` as a mass, so we reject it explicitly.

```
mass = 2.0      # kg
angle = 30.0    # deg
 μ_s = 0.60
 μ_k = 0.40
g = 9.81        # m/s^2
if !(mass isa Real) || mass isa Bool || !isfinite(mass) || mass <= 0
    throw(ArgumentError("mass must be finite, positive, and non-Boolean
   "))
elseif !isfinite(angle) || !(0 <= angle < 90) || μ_k < 0 || μ_k > μ_s
    throw(ArgumentError("use 0 <= angle < 90 and 0 <= μ_k <= μ_s"))
end
 θ = deg2rad(angle)   # rad
normal = mass * g * cosθ()   # N
needed_static = mass * g * sinθ()   # N
if needed_static <= μ_s * normal
    state = "stuck"
```

```
    acceleration = 0.0   # m/s^2
else
    state = "sliding"
    acceleration = (-needed_static + μ_k * normal) / mass   # m/s^2
end
println(state, ", a = ", acceleration, " m/s^2")
```

```
stuck, a = 0.0 m/s^2
```

`stuck, a = 0.0 m/s^2`. Here $mg\sin 30^\circ = 9.81$ N and $\mu_s N \approx 10.19$ N, so static friction is sufficient. If we change only `μ_s` to `0.40`, the result is `sliding` with negative acceleration: downhill in our declared convention. Equality belongs to the stuck branch, our explicit threshold convention.

2.7 Recitation: a pull can reverse impending motion

We add a known uphill pull P to the incline model. Before choosing a friction sign, we calculate the friction-free tendency $T = P - mg\sin\theta$. Positive T means impending uphill motion; negative T means impending downhill motion. Friction opposes that tendency, and static friction can hold whenever $|T| \le \mu_s N$.

We arrive able to read an `if`/`elseif`/`else` chain and convert degrees to radians. In the short guided portion we predict the direction, trace one branch, and test $T = 0$. We then start the separate wall-clearance homework. The takeaway is a repeatable method: validate inputs, state a physical condition, calculate the applicable case, and check its boundary.

2.8 Core homework: does a projectile clear a wall?

We launch from ground level with speed v_0, angle θ, and gravity magnitude $g > 0$, with $+x$ toward the wall and $+y$ upward. In the uniform-gravity, no-drag model,

$$v_x = v_0\cos\theta, \quad t_w = x_w/v_x, \quad y_w = v_0\sin\theta\, t_w - \tfrac{1}{2}gt_w^2, \quad v_{y,w} = v_0\sin\theta - gt_w.$$

We require $v_x > 0$ before dividing. We compare y_w with the wall height, and use $v_{y,w}$ to classify rising or falling. Equality in height counts as clearing; zero vertical velocity belongs to the published falling category. The computed free-flight height may be negative: for a non-negative wall height this means failure to clear, not motion physically continuing through the ground.

We first write a prediction for one ordinary case and one boundary. HW2 is a complete program submitted in CodeRunner: it prints the five input prompts, reads the values with `readline()`, validates the physical domain, and prints one result line for landing before the wall or three result lines otherwise. CodeRunner supplies five input lines and compares the complete printed output, including the prompts. We round the displayed height to four decimal places while using the full-precision value for the state and wall-interaction decisions. The clearance and vertical-state decisions remain separate: a projectile can clear while falling. Neither student-defined functions nor loops are needed.

2.9 Optional bonus: units and AC phasors

This bonus is optional, is not a prerequisite for later work, and is not examined. It is worth **one-third of a final-grade point**.

The self-contained reading develops complex numbers, magnitude, phase, conjugation, and Euler's relation before using `cis`. We choose peak amplitudes and the time convention

$\exp(+i\omega t)$, set the voltage phase to zero, build $Z = |Z|\exp(i\phi_Z)$, and calculate $I = V/Z$. The measurable current is the real part of $I\exp(i\omega t)$; the complex number is a representation, not an additional measured current.

We follow the supplied Unitful examples, verify that voltage divided by resistance has current units, and check a resistor with zero phase before trying a quarter-cycle phase. We then submit the fragment assigning the complex current in amperes and run the published equivalent-unit and invalid-input checks. We explain why a positive impedance phase gives a negative current phase. This is a single-frequency linear steady-state model; switching transients and nonlinear components are outside its scope.

2.10 Going further: total internal reflection

We use the supplied Snell-law recipe $n_1 \sin\theta_1 = n_2 \sin\theta_2$, with positive refractive indices and angles measured from the interface normal. For $n_1 > n_2$, incidence beyond $\theta_c = \arcsin(n_2/n_1)$ gives total internal reflection. We classify below-critical, critical, and above-critical examples before evaluating an inverse sine; blindly taking `asin` of a number greater than one is not a physical calculation.

We sketch repeated ideal reflections in a straight fiber and vary the launch angle. We record which choices remain guided and explain the normal used at each interface. Any supplied tracing loop is an optional preview, not Week 2 core work. Waveguide modes, dispersion, bending losses, and absorption require more physics than this ray model. This investigation is separate from the phasor bonus and is neither examined nor a prerequisite.



3. Vectors, Loops, and Sampled Motion

Many physics questions ask the same calculation at many times or positions. We now make that repetition explicit and store its history in vectors. A vector keeps an ordered collection of values; a loop repeats a block of instructions; together they give us a way to step a model forward through time. Our goal is to understand which instructions repeat, what changes on each step, and what evidence tells us to stop. We call existing tools, but defining our own functions belongs to Week 5.

3.1 Vectors store ordered histories

3.1.1 Creating vectors

A vector is an ordered collection, written with square brackets. We can write a vector directly, start an empty one, or ask for a ready-made vector of zeros or ones:

```
v = [10, 20, 30, 40]
empty_int = Int[]
println(v)
println(empty_int)
println(zeros(5))
println(ones(4))
println(collect(1:5))
println(range(0, 1; length=6))
```

```
[10, 20, 30, 40]
Int64[]
[0.0, 0.0, 0.0, 0.0, 0.0]
[1.0, 1.0, 1.0, 1.0]
[1, 2, 3, 4, 5]
0.0:0.2:1.0
```

The expression `1:5` is a *range*, not a vector: it describes the values 1 through 5 without storing them. We call `collect` when we want an actual vector. The range `0:0.2:1.0` produced by `range(0, 1; length=6)` appears as a range because six points spaced evenly from 0 to 1 are 0.2 apart. `zeros(5)` gives a vector of five floating-point zeros; `ones(4)` gives four ones. `Int[]` is an empty vector whose entries will be integers. Julia arrays are mutable: we can change entries or the length after creation, as we will see shortly.

3.1.2 Indexing and slicing

Julia's ordinary vectors start at index 1. An index identifies a position, not the value stored there; `end` inside an indexing expression identifies the last position.

```
v = [10, 20, 30, 40, 50]
println(v[1])
println(v[end])
println(v[2:4])
println(v[2:end])
v[2:3] = [99, 88]
println(v[[1, 3, 5]])
indices = [2, 3]
println(v[indices])
println(v)
```

```
10
50
[20, 30, 40]
[20, 30, 40, 50]
[10, 88, 50]
[99, 88]
[10, 99, 88, 40, 50]
```

A slice such as `v[2:4]` keeps both endpoints. We can assign to a slice, and we can select several positions at once by handing the index a vector of positions. The functions `length(v)`, `firstindex(v)`, and `lastindex(v)` report the number of entries and the first and last valid positions.

A slice is a copy; an alias is the same vector

For an ordinary vector, `part = height[2:3]` copies those entries. Changing `part[1]` does not change `height`. In contrast, `same = height` makes a second name refer to the same vector; changing `same[1]` changes that shared object. `copy(a)` makes a genuine, independent copy. A *view* shares storage with the original without copying: `@view a[1:2]` lets us read and write the first two entries of `a` through a new name. The `@` symbol marks a macro, a small built-in transformation of the expression that follows; we use it here only to ask for the view, and we do not need to write macros ourselves.

```
a = [1, 2, 3]
b = a
c = a[1:2]
d = copy(a)
e = @view a[1:2]
a[1] = 2
println("a[1]=", a[1], " b[1]=", b[1], " c[1]=", c[1], " d[1]=", d[1], "
    e[1]=", e[1])
```

```
a[1]=2 b[1]=2 c[1]=1 d[1]=1 e[1]=2
```

The alias `b` and the view `e` follow the change; the copy `d` and the slice `c` do not. Which behavior we want depends on whether we mean the same storage or a snapshot of it.

3.1.3 Changing a vector

A trailing `!` in a function name is Julia's naming convention for a mutating operation: it may change the argument itself. `push!` appends one value, `append!` adds every element of a collection, and `pop!` removes and returns the last element. `insert!` and `deleteat!` change a chosen position.

```
v = [1, 2, 3]
push!(v, 4); println(v)
append!(v, [5, 6]); println(v)
pop!(v); println(v)
insert!(v, 2, 99); println(v)
deleteat!(v, 2); println(v)
w = vcat(v, [7, 8]); println(w); println(v)
```

```
[1, 2, 3, 4]
[1, 2, 3, 4, 5, 6]
[1, 2, 3, 4, 5]
[1, 99, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5, 7, 8]
[1, 2, 3, 4, 5]
```

Note the difference: `push!` adds one object, `append!` adds each element of a collection, and `vcat` builds a new concatenated vector while leaving `v` unchanged. In the loop examples ahead we will grow histories with `push!`, which is the natural way to store a sequence of samples as we compute them.

3.1.4 Vector arithmetic and useful functions

Arithmetic on vectors works entry by entry, and some functions summarize a whole vector:

```
a = [1.0, 2.0, 3.0]
b = [4.0, 5.0, 6.0]
println(a + b)
println(2a)
println(sum(a)); println(maximum(a)); println(minimum(a)); println(
   length(a))
println(a .* b)
println(a .^ 2)
println(sqrt.(a))
using LinearAlgebra
println(dot(a, b))
println(round(norm([1,1]); digits=4))
```

```
[5.0, 7.0, 9.0]
[2.0, 4.0, 6.0]
6.0
3.0
1.0
3
[4.0, 10.0, 18.0]
[1.0, 4.0, 9.0]
[1.0, 1.4142135623730951, 1.7320508075688772]
32.0
1.4142
```

The dot marks an element-by-element operation: `a .* b` multiplies matching entries, while `a * b` would be an error for these shapes. Dotted operators act elementwise, so `a .^ 2` squares each entry and `sqrt.(a)` applies the square root to each entry. `sum`, `maximum`, `minimum`, and `length` report one number for the whole vector. The standard module `LinearAlgebra` supplies the dot product `dot(a, b)` and the length `norm(a)`; like `Statistics`, it is part of Julia and needs no installation.

Quick exercise

Create a vector of five values, replace its third entry, append two more values, and compute its mean using `sum(v) / length(v)`. Then check the mean with `using Statistics; mean(v)`.

3.2 A planned repetition: the `for` loop

A loop repeats a block of instructions. A `for` loop visits a supplied collection of values in order: `for value in collection`, a body, and `end`. Before running it, we list the values the loop variable will take. A comment starts with `#`; it explains a choice to a reader and is not executed.

```
for n in 1:3
    println(n)
end
```

```
1
2
3
```

The range `1:3` includes both ends. The more general form `start:step:stop` visits values separated by the step without going beyond the stop. We use a negative step to count downward. The stop itself is included only when the spacing reaches it.

```
for time in 0:2:10   # s
    println("time = $time s")
end
```

```
time = 0 s
time = 2 s
time = 4 s
time = 6 s
time = 8 s
time = 10 s
```

The dollar sign inserts a value into a string. For a calculation rather than a single name we use `$(expression)`, as in `println("square = $(n^2)")`.

There are several ways to visit a vector's entries. The loop variable can take integer values, the values themselves, valid indices, or an index–value pair:

```
v = [10, 20, 30]
for i in 1:3
    println(i)
end
for x in v
    println(x)
end
```

```
for i in eachindex(v)
    println("index = ", i, ", value = ", v[i])
end
for (i, x) in enumerate(v)
    println(i, " => ", x)
end
```

```
1
2
3
10
20
30
index = 1, value = 10
index = 2, value = 20
index = 3, value = 30
1 => 10
2 => 20
3 => 30
```

Which style we choose depends on what the body needs. When only values matter, `for x in v` is clearest. When we index or modify entries, `eachindex(v)` supplies the valid indices for that collection, so we do not have to assume a particular index range. `enumerate` supplies a counter alongside each element; its counter is the ordinary 1-based position here, which is all we need.

An accumulator is a value carried between repetitions

To add several values we initialize a total, add one contribution, and keep the new total for the next step. We must initialize outside the repeated body; resetting it inside would discard earlier contributions. A counter records how many steps we have performed, while an accumulator records their combined result. The examples below group their statements in a `begin ... end` block, so each example reads as one unit and can be pasted and run as is.

```
begin v = [3, 5, 8, 2]
    total = 0
    for x in v
        total += x
    end
    println(total)
    println(sum(v))
end
```

```
18
18
```

The loop reproduces `sum(v)` one addition at a time. This is the skeleton of almost every accumulation in this course: start with a neutral value, combine one item per repetition, and keep the result for the next step. A factorial is the same pattern with multiplication instead of addition:

```
begin f = 1
    for n in 1:5
```

```
        f = f * n
    end
    println(f)
end
```

```
120
```

A counter is an accumulator of steps that satisfy a condition:

```
begin v = [-2, 3, 0, 7, -1]
    count_positive = 0
    for x in v
        if x > 0
            count_positive += 1
        end
    end
    println(count_positive)
end
```

```
2
```

Finding a maximum without `maximum` uses an accumulator that only improves:

```
begin v = [4, 9, 2, 7]
    largest = v[1]
    for x in v
        if x > largest
            largest = x
        end
    end
    println(largest)
end
```

```
9
```

Building a vector of running totals combines an accumulator with `push!`:

```
begin v = [2, 3, 5, 7]
    running = 0
    cumulative = Int[]
    for x in v
        running += x
        push!(cumulative, running)
    end
    println(cumulative)
    println(cumsum(v))
end
```

```
[2, 5, 10, 17]
[2, 5, 10, 17]
```

Here the vector `cumulative` records the state at every step, not just the final one. This is the pattern we will use to store a trajectory: push each new sample as it is computed, and compare with a known result.

A common vector calculation is the dot product, which also uses an accumulator over matching indices:

```
using LinearAlgebra
begin
    a = [1.0, 2.0, 3.0]
    b = [4.0, 5.0, 6.0]
    s = 0.0
    for i in eachindex(a, b)
        s += a[i] * b[i]
    end
    println(s)              # 32.0
    println(dot(a, b), " ", ·ab)   # 32.0 32.0
end
```

```
32.0
32.0 32.0
```

The two vectors must have the same length for this loop to make sense; that is exactly what `eachindex(a, b)` requires. Accumulating a sum with `s += a[i] * b[i]` is the manual version of the dot product.

3.3 Summaries: mean and sample standard deviation

Location and spread answer different questions

For n observations q_i, the mean is $\bar{q} = \sum_i q_i/n$. The corrected sample standard deviation is

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(q_i - \bar{q})^2}, \qquad n \geq 2.$$

Both have the units of q. The mean locates the sample; the standard deviation measures its spread. Neither a smaller printed decimal nor a small spread rules out systematic measurement error. Interpreting $s/\sqrt{n}$ as uncertainty of a mean requires additional assumptions about independent repeated measurements.

`Statistics` is a standard-library module supplied with Julia, so it needs no package installation. The statement `using Statistics` makes its `mean` and `std` functions available:

```
using Statistics
measurements = [1.0, 2.0, 3.0]   # m
println(mean(measurements))
println(std(measurements))
```

```
2.0
1.0
```

Its default `std` uses the correction above, and both functions work on a whole vector. A trajectory is not automatically a set of independent repeated measurements; a spread of positions does not by itself estimate instrumental uncertainty.

3.4 A condition-controlled repetition: `while`

A `while` loop checks its condition before every iteration. We identify its initialization, condition, update, and termination argument. It may execute zero times. `break` leaves the nearest loop immediately; in a bounded search it allows us to stop at the first success.

```
for n in 1:10
    if n^2 >= 20
        println("first square at least 20 is $(n^2)")
        break
    end
end
```

```
first square at least 20 is 25
```

Stopping is part of the scientific model

We distinguish the first successful sample from the exact time a continuous event occurs. We retain a maximum-step bound when termination has not been established. A missing update can create an infinite loop; a wrong inequality can skip an initial or boundary event. Sentinel-input programs and advanced `continue` exercises are optional later reading, not prerequisites here.

3.5 From the derivative to an Euler update

Projectile motion: laws and limitations

We take $+x$ right and $+y$ upward. A point-like projectile in approximately uniform gravity, without air resistance, has $\dot{x} = v_x$, $\dot{y} = v_y$, $\dot{v}_x = 0$, and $\dot{v}_y = -g$. Positions are in metres, velocities in m/s, time in seconds, and $g > 0$ in m/s^2. Horizontal velocity must stay constant. Drag, bounce, changing gravity, and Earth's curvature are omitted; a numerical step cannot recover these missing effects.

Old values determine the next state

The derivative definition gives $[y(t + \Delta t) - y(t)]/\Delta t \approx v_y(t)$ over a small step. Thus explicit Euler uses

$$x_{n+1} = x_n + v_{x,n}\Delta t, \quad y_{n+1} = y_n + v_{y,n}\Delta t, \quad v_{y,n+1} = v_{y,n} - g\Delta t.$$

Updating the position with the new velocity instead gives a different algorithm. The exact height in this constant-gravity example includes the additional term $-g\Delta t^2/2$ per step. Euler neglects that term; repeated small omissions produce a global timestep error.

We store position and velocity as two-component vectors, $\mathbf{r} = (x, y)$ and $\mathbf{v} = (v_x, v_y)$, with $\mathbf{a} = (0, -g)$. A finite difference replaces the derivative over each short interval:

$$\frac{dx}{dt} \approx \frac{x(t + \Delta t) - x(t)}{\Delta t} \quad \Longrightarrow \quad x(t + \Delta t) \approx x(t) + v_x(t)\Delta t.$$

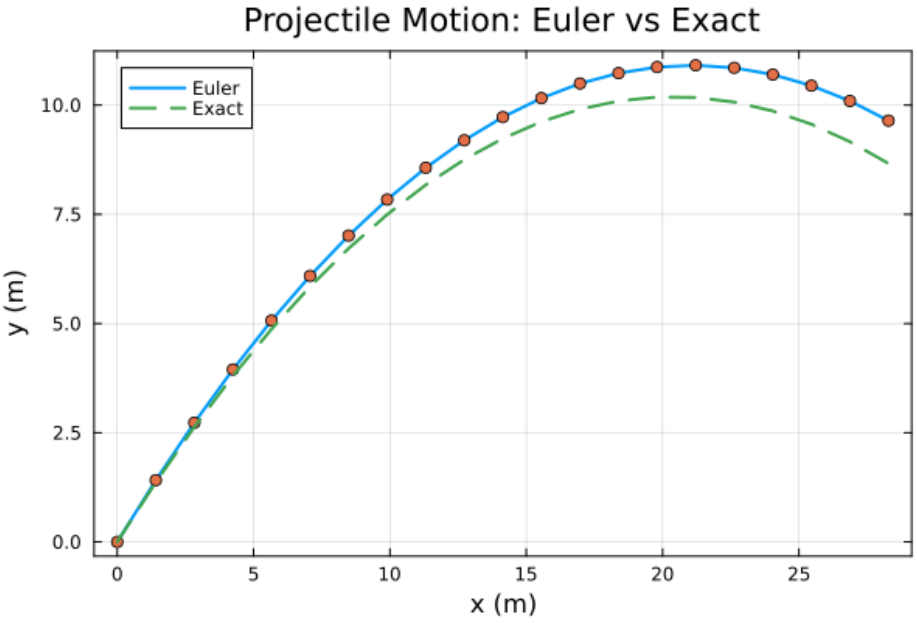
The same reasoning gives $\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_n\Delta t$ and $\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n\Delta t$. Each new state is calculated from the previous one. In components, the velocity updates are $v_{x,n+1} = v_{x,n} + a_{x,n}\Delta t$ and $v_{y,n+1} = v_{y,n} + a_{y,n}\Delta t$.

Let us follow a launch for $T = 2$ s, before it reaches the ground. Four growing vectors store the horizontal and vertical components of both position and velocity. The plot uses these stored samples; we do not integrate a second time to draw it. We calculate the comparison parabola from the initial position and velocity.

The supplied `let ... end` block keeps the simulation and its plot in one local work area. Run the whole block together. We will return to local variables when we introduce functions; for now the wrapper lets the numerical example behave consistently in a script or notebook. The earlier `begin ... end` examples remain interactive exercises. `round(Int, T / dt)` gives the integer step count, and here $T/\Delta t = 20$ exactly. The `Plots` package draws the curves and sample markers; `display(p)` shows the resulting plot.

```
using Plots
let
    trajectory_x = Float64[]
    trajectory_y = Float64[]
    velocities_x = Float64[]
    velocities_y = Float64[]
    g = 9.81;        # gravity in m/s^2
    speed = 20.0;    # initial speed in m/s
    theta = pi / 4;  # initial angle in radians
    dt = 0.1;        # time step in seconds
    T = 2.0          # total time in seconds
    r = [0.0, 0.0]   # m
    v = [speed * cos(theta), speed * sin(theta)]   # m/s
    a = [0.0, -g]    # m/s^2
    N = round(Int, T / dt)  # number of time steps
    trajectory_x = [r[1]]
    trajectory_y = [r[2]]
    velocities_x = [v[1]]
    velocities_y = [v[2]]
    for step in 1:N
        r = r + v * dt
        v = v + a * dt
        push!(trajectory_x, r[1])
        push!(trajectory_y, r[2])
        push!(velocities_x, v[1])
        push!(velocities_y, v[2])
    end
    println("Euler final position: ", round(r[1]; digits=4), ", ", round
   (r[2]; digits=4))
    r0 = [0.0, 0.0]
    v0 = [speed * cos(theta), speed * sin(theta)]
    p = plot(title="Projectile Motion: Euler vs Exact", xlabel="x (m)",
   ylabel="y (m)", frame=:box)
    p = plot!(p, trajectory_x, trajectory_y, label="Euler", lw=2)
    p = scatter!(p, trajectory_x, trajectory_y, label=:none, lw=2)
    exact_trajectory_x = Float64[]
    exact_trajectory_y = Float64[]
    ts = 0:dt:T
    for t in ts
        push!(exact_trajectory_x, r0[1] + v0[1] * t + 0.5 * a[1] * t^2)
        push!(exact_trajectory_y, r0[2] + v0[2] * t + 0.5 * a[2] * t^2)
    end
    p = plot!(p, exact_trajectory_x, exact_trajectory_y, label="Exact",
   lw=2, ls=:dash)
    display(p)
end
```

```
Euler final position: 28.2843, 9.6453
```



The markers show the stored Euler samples, and the dashed line shows the exact constant-gravity path. At $t = T$, the Euler height is 9.6453 m, compared with the exact height 8.6643 m. The difference is $gT\Delta t/2 = 0.981$ m. The lines joining samples are visual guides. The next example asks how that difference changes as we shorten the timestep.

3.6 Refinement: compare with a known answer

For initial position $\mathbf{r}_0$ and velocity $\mathbf{v}_0$, the exact motion under constant acceleration is

$$\mathbf{r}_{\text{exact}}(t) = \mathbf{r}_0 + \mathbf{v}_0 t + \tfrac{1}{2}\mathbf{a}t^2.$$

Let us run the same integration for several time steps, keeping $T/\Delta t$ integral:

```
using LinearAlgebra
let
    g = 9.81; speed = 20.0; theta = pi / 4; T = 2.0
    r0 = [0.0, 0.0]
    v0 = [speed * cos(theta), speed * sin(theta)]
    a = [0.0, -g]
    for dt in (0.2, 0.1, 0.05, 0.02)
        r = copy(r0); v = copy(v0)
        N = round(Int, T / dt)
        for step in 1:N
            r = r + v * dt
            v = v + a * dt
        end
        exact = r0 + v0 * T + 0.5 * a * T^2
        println("dt = $dt, error = ", round(norm(r - exact); digits=4),
   " m")
    end
end
```

```
dt = 0.2, error = 1.962 m
dt = 0.1, error = 0.981 m
dt = 0.05, error = 0.4905 m
dt = 0.02, error = 0.1962 m
```

Halving Δt halves the position error. For constant acceleration and explicit Euler, the velocity at the discrete times is exact up to floating-point rounding, while the position carries a systematic error:

$$\mathbf{r}_N - \mathbf{r}_{\text{exact}}(T) = -\tfrac{1}{2}\mathbf{a}\,T\Delta t.$$

For downward gravity, the numerical trajectory lies *above* the exact trajectory at a fixed time, and the error magnitude is $gT\Delta t/2$, exactly the values printed above.

Why does the error scale with Δt? Each Euler position update omits the $\frac{1}{2}a(\Delta t)^2$ term, so the local position error is proportional to $(\Delta t)^2$. There are $N = T/\Delta t$ steps. In this constant-acceleration example the omitted contributions add coherently, giving a global error proportional to $N(\Delta t)^2 = T\Delta t$. This is *first-order convergence*. The simulation is not exact physics, but the error shrinks predictably, and we can see it shrink.

Closing question: What happens to the error if we make the time step ten times smaller? Test it numerically before explaining.

Higher-order integration: a first look (optional)

Euler's first-order error is not the end of the story. A better idea: take a half step, evaluate the derivative at that halfway point, and use it to take the full step:

$$k_1 = f(y_n), \qquad k_2 = f(y_n + \tfrac{1}{2}\Delta t\, k_1), \qquad y_{n+1} = y_n + \Delta t\, k_2.$$

A method using one mid-interval derivative is a second-order method; for constant acceleration it happens to be exact. Taking more cleverly placed samples of the derivative gives still higher orders, up to the classic fourth-order Runge–Kutta method ("RK4"). These schemes are optional background, not something we need to write in this course.

3.7 Recitation: first sampled cooling time

We use Newton's cooling model, $\dot{T} = -k(T - T_{\text{room}})$, for a body with approximately uniform internal temperature in a fixed environment. The rate $k > 0$ is in s^{-1}; temperature differences may be measured in kelvin or degrees Celsius. We assume heat loss is approximately proportional to that difference, rather than claiming the model covers evaporation or large radiative effects.

The recitation notebook supplies the physical update. We identify initialization, the condition, the update, and the counter, then use a `while` loop to find the first sample at or below a threshold. Equality counts as reaching the threshold. We predict what happens if the initial sample already qualifies, and why a threshold below room temperature cannot be reached by monotone cooling from above. After this 20-minute discussion we start the hammer homework, whose loop stops at the first sample below the ground.

3.8 Core homework: a hammer throw with air resistance

We simulate a two-dimensional hammer throw from the origin with quadratic air drag. The model per unit mass is

$$\dot{x} = v_x, \quad \dot{y} = v_y, \quad \dot{v}_x = -bvv_x, \quad \dot{v}_y = -g - bvv_y, \qquad v = \sqrt{v_x^2 + v_y^2},$$

with $g = 9.81$ m/s^2 and a drag coefficient $b \approx 0.001$ m^{-1}. The program reads the initial speed, the launch angle in degrees, the total time T, the time step Δt, and b, then advances the state with explicit Euler and stores aligned histories of x, y, v_x, v_y. It stops at the first proposed sample with $y < 0$, or after T, and prints the horizontal distance reached.

We first sketch the expected trajectory and state the units and signs. Then we construct the vectors and the loop, taking care to update position from the old velocity. We check the first update by hand and verify that each vector grows by one entry per step, that the horizontal velocity decays only through drag, and that the vertical acceleration includes both gravity and the drag term. We run the program with the given example, compare the printed distance with a hand-calculated estimate, and refine Δt to see how the sampled distance moves. We save the trajectory plot as an artifact. The report must state which rule stopped the loop: the ground crossing or the end of the simulation window. The optional extension on the sheet, testing whether 45° still maximizes the range, is not required. Function definitions remain Week 5 material.

3.9 Optional bonus: three integration schemes

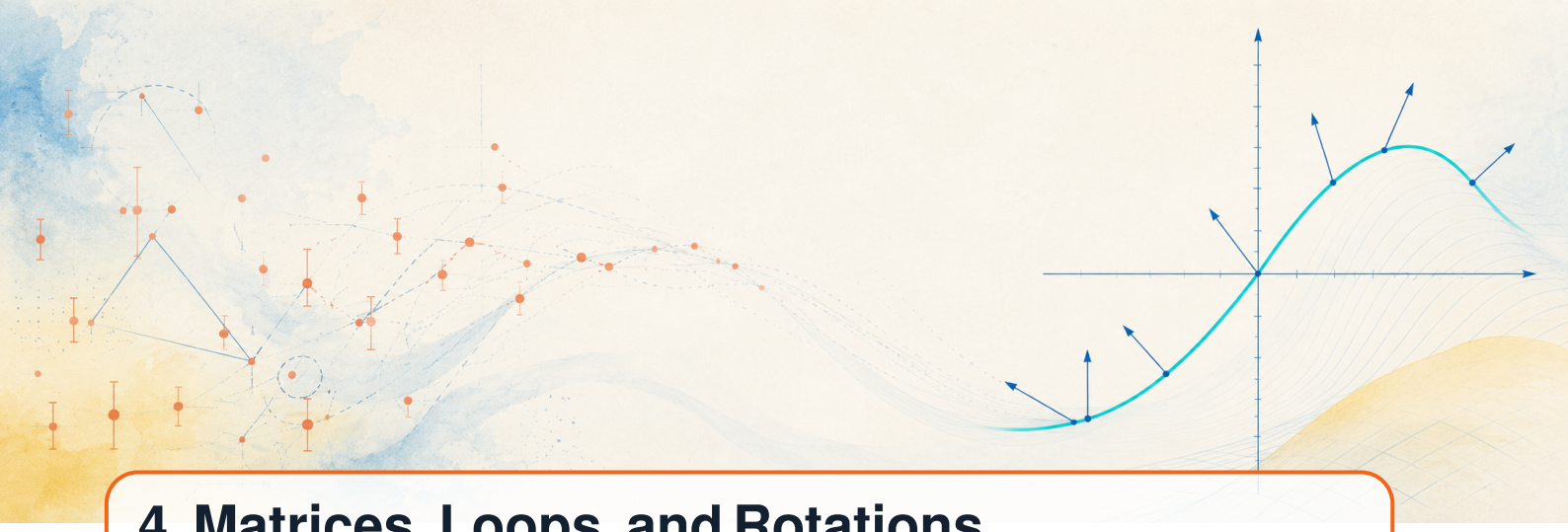
This bonus is optional, is not a prerequisite for later work, and is not examined. It is worth **one-third of a final-grade point**.

We compare forward Euler, Euler–Cromer, and the velocity-Verlet (midpoint) scheme on a free fall by tracking the mechanical energy per unit mass. We read the final time, the time step, the initial height, and the initial speed, run all three schemes over the same grid, and print the three final specific energies. We then plot the specific energy against time for all three schemes together with the exact conserved value. Two short theory questions ask us to derive the per-step energy change for each scheme and to say which scheme is preferable and why.

3.10 Going further: from one walk to diffusion

One path can wander back toward its origin. Diffusion is an ensemble statement: for independent unbiased unit steps, the mean-square displacement satisfies $\langle r_n^2 \rangle = n$ in lattice units, since cross terms average to zero. If one step lasts τ and has length ℓ, the corresponding two-dimensional diffusion coefficient is $D = \ell^2/(4\tau)$.

Using a supplied ensemble recipe, we compare a small and a larger collection of walks, plot mean-square displacement against step number, and examine fluctuations around the expected straight line. We explain why the square of the mean displacement is a different quantity and can stay near zero. We record the generator setup and ensemble size with the figure. This investigation is optional and unexamined; the supplied ensemble machinery is not a prerequisite for core work.



4. Matrices, Loops, and Rotations

Many quantities in physics are not single numbers but organized tables of numbers. A matrix stores such a table, and a rotation is one of the most important transformations we can apply to it. We learn to read a matrix and to ask about its shape, then we use loops to fill matrices and to combine them, and finally we use matrices to rotate shapes and vectors. Throughout the chapter we write the arithmetic ourselves, one entry at a time, so that we understand what the built-in operations are actually doing. Our own functions arrive in Week 5; here everything is written with loops and `let` work areas.

4.1 Matrix literals and shape

A matrix is a rectangular array of numbers arranged in rows and columns. When we write a matrix in Julia, spaces separate the entries of one row, and a semicolon separates the rows:

```
A = [1 2 3; 4 5 6]      # a 2-by-3 matrix: 2 rows, 3 columns
println(A)
println(size(A))         # (2, 3)
println(size(A, 1))      # 2   (number of rows)
println(size(A, 2))      # 3   (number of columns)
println(length(A))       # 6   (total number of entries)
```

```
[1 2 3; 4 5 6]
(2, 3)
2
3
6
```

The matrix `A` has two rows and three columns, so its *shape* is "2-by-3". `size(A)` returns the pair of dimensions, `size(A, 1)` the number of rows, and `size(A, 2)` the number of columns. The total number of entries is the product of the two dimensions, which `length` reports. Knowing the shape is usually the first thing we check, because two matrices can only be combined when their shapes are compatible.

4.1.1 Indexing a matrix

We locate a single entry by its row index first, then its column index:

```
println(A[2, 3])          # 6
println(A[1, :])          # [1 2 3]   the first row (a 1-by-3 matrix)
println(A[:, 2])          # [2, 5]    the second column (a vector)
```

```
6
[1 2 3]
[2, 5]
```

`A[2, 3]` is the entry in row 2 and column 3. The colon selects everything along that axis: `A[1, :]` is the whole first row, and `A[:, 2]` is the whole second column. Notice the difference in what is printed. A row keeps its horizontal layout, while a column comes out as an ordinary vector. A matrix with a single column is still a matrix, and Julia prints it that way.

Rows first, columns second

Every indexing expression names the row before the column: `A[i, j]` means row i, column j. The colon `:` means "every index along this axis". Slicing a row or a column always returns something with the correct orientation, so the result can go straight into a product or a plot.

4.2 while loops: repeat until a condition turns false

Some repetitions have no known length in advance. A `while` loop repeats for as long as its condition is true, testing the condition before every iteration. Unlike a `for` loop it carries no counter of its own: something inside the body must change the condition, or the loop never ends.

```
let
    n = 0
    while n < 5
        n += 1
        println(n)
    end
end                       # prints 1, 2, 3, 4, 5
```

```
1
2
3
4
5
```

Here `n` is the counter, `n += 1` is the update, and `n < 5` is the condition. The loop tests the condition, runs the body, tests again, and stops at the moment the test fails.

4.2.1 Sentinel input: a class average

A frequent use of a `while` loop is reading input until a special *sentinel* value appears. Below we read grades until the user enters -1. The sentinel marks the end of the input; it is not a grade.

```
let
    sum_grades = 0      # total of the grades
    n_grades = 0        # how many grades were entered
    print("Enter grade (-1 to stop): ")
```

```
    grade = parse(Int, readline())
    while grade != -1
        sum_grades += grade
        n_grades += 1
        print("Enter grade (-1 to stop): ")
        grade = parse(Int, readline())
    end
    if n_grades != 0
        avg = sum_grades / n_grades
        println("Class average is ", round(avg; digits = 2),
                "; number of grades is ", n_grades)
    else
        println("No grades were entered")
    end
end
```

The first read happens before the loop and every iteration ends by reading again. If the user enters -1 immediately, the loop never runs and we report that no grades were entered instead of dividing by zero.

A sentinel is data, not a mistake

A sentinel value marks the end of a stream of inputs, so it must be distinguishable from real data. Here -1 works because grades lie between 0 and 100. The `while` condition tests for the sentinel; it does not count iterations.

4.2.2 Validating input

A second standard pattern keeps asking until the answer is acceptable. The loop continues while the input is bad, so the condition is the negation of what we want:

```
let
    print("Please enter a positive number 20..100: ")
    flush(stdout) # Make sure the prompt appears before waiting for
   input.
    number = parse(Int, readline())
    while number <= 20 || number >= 100
        print("A positive number 20..100, try again: ")
        number = parse(Int, readline())
    end
    println("Your positive number is ", number)
end
```

The condition `number <= 20 || number >= 100` reads as "still bad". Only a value strictly between 20 and 100 stops the loop. Look at the boundaries: with `<=` and `>=`, exactly 20 and exactly 100 are rejected. Accepting them would require `<` and `>`. The `flush(stdout)` call pushes the prompt out before the program waits, which matters in terminals that buffer output.

4.2.3 A guessing game

The guessing game combines both ideas. The computer picks a number, and we keep reading guesses, receiving a hint each time, until one matches:

```
let
    number = rand(1:16)
    print("please guess my number between 1 and 16 ")
```

```
    flush(stdout) #
    guess = parse(Int, readline())
    while guess != number
        if number > guess
            print("My number is bigger, please try again ")
        else
            print("My number is smaller, please try again ")
        end
        flush(stdout)
        guess = parse(Int, readline())
    end
    println("You guessed it! The number is ", number)
end
```

The loop may run zero times, if the first guess is right, or many times. Each wrong guess produces a hint that narrows the search, which is why a player rarely needs more than a handful of tries out of sixteen possibilities.

4.2.4 Timing a while loop: the harmonic series

The harmonic series $1 + \frac{1}{2} + \frac{1}{3} + \dots$ grows very slowly, roughly like $\ln n$. Reaching even a modest sum therefore takes many terms, which makes it a good way to feel how fast Julia runs. We keep adding terms until the sum exceeds a limit, and we time the loop:

```
let
    print("Enter positive limit: ")
    flush(stdout)
    limit = parse(Float64, readline())

    max_terms = 100_000_000   # safety cap on the number of terms
    t1 = time_ns()
    series_sum = 0.0
    n = 0
    while series_sum <= limit
        n += 1
        series_sum += 1.0 / n
        if n >= max_terms
            println("Limit ", limit, " too large: stopped after ", n,
                    " terms without exceeding it.")
            break
        end
    end
    t2 = time_ns()

    println("Number of terms needed: ", n)
    println("Elapsed seconds: ", round((t2 - t1) / 1.0e9; digits = 3))
    # For limit 10: 12367 terms (instant).
    # For limit 20: about 272,400,600 terms — more than the cap, so the
    loop
    # stops at 100,000,000 terms and prints that the limit was too large.

end
```

For a limit of 10 the answer is 12367 terms, which finishes instantly. For a limit of 20 we would need about 272 million terms, more than our safety cap of one hundred million, so the loop stops early and says that the limit was too large. That `if n >= max_terms` guard

with `break` is a habit worth forming: whenever a loop might run for a very long time, give it a bound and decide what to do when the bound is reached. The timings come from `time_ns()`, which reads a nanosecond clock, so dividing the difference by 10^9 gives seconds.

Stopping is part of the design

A `while` loop needs three things: an initialization, a condition that can become false, and an update that moves the condition. Missing any one of them gives an infinite loop. A maximum-iteration bound protects against accidents and turns an unbounded search into a finite one.

These interactive examples run more reliably as standalone files than cell by cell in an editor, so the four input examples are also collected in the `week04_readline_examples` folder inside `examples`. From the REPL, after moving into that `examples` folder, you can `include("week04_readline_examples/class_average_sentinel.jl")`, and likewise for the other three files.

4.3 Matrix–vector and matrix–matrix products

A matrix can act on a vector or on another matrix, and Julia writes these products with the ordinary *:

```
M = [1 2; 3 4]
v = [5, 6]
println(M * v)           # [17, 39]   matrix * vector: linear
   combination of columns
println(M .* M)          # [1 4; 9 16]  elementwise multiplication (
   dotted)
println(M * M)           # [7 10; 15 22]  true matrix product
```

```
[17, 39]
[1 4; 9 16]
[7 10; 15 22]
```

The first line is a matrix–vector product: the columns of `M` are combined with the entries of `v` to give a new vector. The second line is *elementwise* multiplication, written with a dot: each entry of `M` is multiplied by the entry in the same position. The third line is a genuine matrix product, and it is the one we will rebuild by hand with loops in a moment.

Matrix multiplication is not entry-by-entry

The entry in row i and column j of the product `A * B` is the dot product of row i of `A` with column j of `B`:

$$(AB)_{ij} = \sum_k A_{ik} B_{kj}.$$

For this sum to make sense the inner dimensions must agree: the number of columns of `A` must equal the number of rows of `B`. Elementwise multiplication `A .* B` instead pairs entries in the same position and requires identical shapes. The dotted version is a different operation and never a substitute for the product.

4.4 Nested loops: a multiplication table

A loop repeats a block of instructions. A *nested* loop places one loop inside another, so the inner loop runs to completion for every value of the outer one. Nested loops are the natural way to fill a matrix row by row. The simplest example is a 10×10 table of products, where each entry `table[i, j]` holds `i * j`:

```
let
    table = zeros(Int, 10, 10)
    for i in 1:10
        for j in 1:10
            table[i, j] = i * j
        end
    end
    println(table[3, 4])       # 12
    println(table[10, 10])     # 100
    println(table[3, 1:5])     # [3, 6, 9, 12, 15]   (row 3)
    println(table)
end
```

```
12
100
[3, 6, 9, 12, 15]
```

Trace the order of the loops. The outer loop sets $i = 1$ and the inner loop sweeps $j = 1, \ldots, 10$, filling the whole first row. Then the outer loop moves to $i = 2$ and the inner loop fills the second row. The inner loop is the one that changes fastest, which is exactly how we read a table: left to right, row by row. The final `println(table)` above that output prints the entire matrix in Julia's compact notation, one very long line with rows separated by semicolons, which is correct but almost impossible to read.

Printing a matrix readably (optional)

The plain printed matrix is enough for our purposes, but wide matrices become hard to read. If you want aligned columns, the package `PrettyTables.jl` provides `pretty_table(table)`, which draws a bordered table. It is not part of the course environment, it is not needed for any exercise, and installing extra packages is outside the scope of this course; the raw matrix print is what we use.

4.5 Nested loops: matrix multiplication

Now we can reproduce the product ourselves. Each output entry is a dot product of one row with one column, so one double loop visits every output entry while an inner loop accumulates the sum over the shared dimension. When a matrix is long enough that a single line would be cramped, we are free to let it run over several lines, aligning the rows so that the shape stays visible:

```
let
    A = [1.0 2.0 3.0;
         4.0 5.0 6.0]   # 2-by-3
    B = [1.0 0.0 2.0 1.0;
         0.0 1.0 1.0 0.0;
         2.0 1.0 0.0 1.0]   # 3-by-4
    m, k = size(A)
    k2, n = size(B)
```

```
    @assert k == k2 "inner dimensions must match"

    C = zeros(m, n)
    for i in 1:m          # output row
        for j in 1:n      # output column
            for r in 1:k  # the shared inner dimension
                C[i, j] += A[i, r] * B[r, j]
            end
        end
    end

    println(C)             # 7.0  5.0  4.0  4.0  /  16.0  11.0  13.0
   10.0
    println(A * B)         # same numbers, built by Julia's own matrix
   product
    println(isapprox(C, A * B))   # true
end
```

```
[7.0 5.0 4.0 4.0; 16.0 11.0 13.0 10.0]
[7.0 5.0 4.0 4.0; 16.0 11.0 13.0 10.0]
true
```

This is a triple loop. The outer two loops choose the output entry (i, j), and the innermost adds $A[i,1]B[1,j] + A[i,2]B[2,j] + \dots$ until the shared dimension is used up. Spreading the matrices over several lines changes nothing about the values; it only keeps the row structure readable, and the trailing comment reminds us which shape we wrote. The `@assert` line is a guard: if the inner dimensions disagree, the program stops with a clear message rather than computing nonsense. Agreement with Julia's own `A * B` is the confirmation that we understood the rule.

4.6 Checking whether a matrix is symmetric

A matrix is symmetric when $A_{ij} = A_{ji}$ for every pair of indices: the entry in row i, column j equals the entry in row j, column i. A nested loop can decide this. Only the upper triangle needs inspection, because the diagonal always equals itself:

```
let
    M = [1 2 3;
         2 5 6;
         3 6 9]    # symmetric by construction
    symmetric = true
    for i in 1:size(M, 1)
        for j in i:size(M, 2)    # j starts at i, so we visit the upper
   triangle
            if M[i, j] != M[j, i]
                symmetric = false
            end
        end
    end
    println("Matrix M is symmetric: $symmetric")           # true

    N = [1 2 3;
         4 5 6;
         7 8 9]   # not symmetric
    symmetric = true
```

```
    for i in 1:size(N, 1), j in i:size(N, 2)
        if N[i, j] != N[j, i]
            symmetric = false
        end
    end
    println("Matrix N is symmetric: $symmetric")          # false
end
```

```
Matrix M is symmetric: true
Matrix N is symmetric: false
```

Both matrices are written one row per line, which makes the symmetry of `M` visible at a glance: reading across a row gives the same numbers as reading down a column. The loop checks that property directly. Starting the inner loop at `j in i:…` skips the lower triangle and the diagonal, roughly halving the work. The second loop uses the compact form `for i in …, j in …`, which means the same as two nested loops.

A shortcut is available. The transpose `M'` swaps rows and columns, so a matrix is symmetric exactly when `M == M'`. Applying that test to a nearly symmetric matrix with floating-point entries fails, because round-off perturbs one pair by about 10^{-14}. There `isapprox` with a tolerance is the right tool, as the final block shows.

```
# A shortcut: M' is the transpose. Comparing M with its transpose is the
    same check.
M = [1 2 3; 2 5 6; 3 6 9]
N = [1 2 3; 4 5 6; 7 8 9]
println(M == M')         # true
println(N == N')         # false
```

```
true
false
```

The same comparison in floating point, with a tolerance:

```
let
    P = [1.0 2.0;
         2.0 1.0]
    Q = P + [0.0 1e-14;
             0.0 0.0]   # nearly symmetric, but not exactly
    println(P == P')                # true
    println(Q == Q')                # false  (machine noise breaks exact
    equality)
    println(isapprox(Q, Q'; atol = 1e-12))    # true
    println(maximum(abs.(Q - Q')))            # 1.0e-14
end
```

```
true
false
true
1.0e-14
```

`Q` differs from its transpose by only 10^{-14}, which is well below any physical relevance but enough to make `==` fail. `isapprox` with `atol = 1e-12` accepts it. `maximum(abs.(Q - Q'))` reports the size of the discrepancy directly; the dot in `abs.` broadcasts the absolute value over every entry.

4.7 Rotation matrices in 2D

A matrix that rotates a vector counterclockwise by an angle θ is

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}.$$

With column vectors the rotated point is $R\,\mathbf{p}$. Rotating the vector $(1, 0)$ by $90°$ should turn it into $(0, 1)$:

```
using LinearAlgebra
let
    θ = π / 2
    R = [cosθ() -sinθ();
         sinθ() cosθ()]
    println(R)                          # 0.0  -1.0  /  1.0  0.0
    p = [1.0, 0.0]
    println(R * p)                      # [0.0, 1.0]   (a quarter turn)
end
```

```
[0.0 -1.0; 1.0 0.0]
[0.0, 1.0]
```

The matrix is written with the second row on its own line, which keeps the two rows visible as rows. Julia reports the entries as `6.12e-17` rather than exactly zero because $\cos(\pi/2)$ is not exactly representable in floating point; the printed value rounds to `0.0`. That tiny residue is the reason the next block compares quantities with a tolerance rather than with `==`.

4.7.1 Checking that a matrix really is a rotation

Two properties distinguish a rotation from an arbitrary matrix. A rotation preserves distances, so the separation of two points is unchanged. Its transpose is its inverse, $R'R = I$, and it preserves orientation, so its determinant is 1. We can test the first two directly:

```
let
    p = [1.0, 2.0]
    q = [-0.5, 3.0]
    θ = π / 6
    R = [cosθ() -sinθ();
         sinθ() cosθ()]

    println(isapprox(R' * R, I; atol = 1e-14))   # true  (orthogonal)

    d_before = norm(p - q)         # distance before rotation
    d_after = norm(R * p - R * q)  # distance after rotation
    println(round(d_before; digits = 6))         # 1.802776
    println(round(d_after; digits = 6))          # 1.802776   (same
   distance)
end
```

```
true
1.802776
1.802776
```

`R' * R` comes out as the identity matrix to within 10^{-14}, and the distance between the two rotated points equals the distance between the originals. If we ever construct a transformation that claims to be a rotation, these two checks decide the question quickly.

4.8 Rotating a rectangle in the plane

Rotating a shape means rotating every vertex and joining the results again. A rectangle is stored as a matrix whose rows are its four corners. Below the rectangle is turned by $0°, 30°, 60°, 90°$, and after each turn we print where one corner went:

```
let
    θs = (0.0, 30.0, 60.0, 90.0)
    rect = [0.0 0.0;
            2.0 0.0;
            2.0 1.0;
            0.0 1.0]
    for θ in θs
        R = [cosd(θ) -sind(θ);
             sind(θ) cosd(θ)]
        rotated = zeros(size(rect))  # same shape as rect
        for vertex in 1:4
            corner = [rect[vertex, 1], rect[vertex, 2]]   # the corner
   as a vector
            moved = R * corner                            # the rotated
   corner
            rotated[vertex, 1] = moved[1]
            rotated[vertex, 2] = moved[2]
        end
        println(θ, " = ", θ, "°  corner (2,0) -> [", round(rotated[2, 1];
   digits = 3),
                ", ", round(rotated[2, 2]; digits = 3), "]")
    end
    # θ = 0.0°   corner (2,0) -> [2.0, 0.0]
    # θ = 30.0°  corner (2,0) -> [1.732, 1.0]
    # θ = 60.0°  corner (2,0) -> [1.0, 1.732]
    # θ = 90.0°  corner (2,0) -> [0.0, 2.0]
end
```

Each row of `rect` is one corner, written one per line so the four vertices stay distinguishable. The inner loop reads the two coordinates of one corner with single indices, `rect[vertex, 1]` and `rect[vertex, 2]`, packs them into a two-element vector, and multiplies the matrix by that vector. This is exactly the matrix–vector product of the earlier section, applied once per corner, so no knowledge of slicing is needed. The two coordinates of the result go into the matching row of `rotated`, which `zeros(size(rect))` has already prepared with the same shape. The corner at $(2, 0)$ sweeps around the circle of radius two, passing through $(1.732, 1.0)$ at $30°$ and ending on the y-axis at $90°$, which is what a quarter turn should do. The functions `cosd` and `sind` take degrees directly, avoiding an explicit conversion.

Drawing the copies together makes the rotation easier to see than reading coordinates:

```
using Plots
let
    rect = [0.0 0.0; 2.0 0.0; 2.0 1.0; 0.0 1.0]

    p = plot(title = "Rectangle rotated in the xy plane",
             xlabel = "x", ylabel = "y",
             legend = false, aspect_ratio = :equal)

    for θ in 0:30:330
        R = [cosd(θ) -sind(θ); sind(θ) cosd(θ)]
```

```
        xs = Float64[]; ys = Float64[]
        for vertex in 1:4
            corner = [rect[vertex, 1], rect[vertex, 2]]
            moved = R * corner
            push!(xs, moved[1])
            push!(ys, moved[2])
        end
        # Close the polygon back to its first vertex.
        push!(xs, xs[1]); push!(ys, ys[1])
        plot!(p, xs, ys; lw = 2)
    end

    display(p)
end
```

Two things in this block deserve attention. The outer loop steps through the angles from 0° to 330° in increments of thirty, so each iteration draws one rotated rectangle. Inside, each corner becomes a two-element vector, is rotated with the same matrix–vector product as before, and its two coordinates are collected into `xs` and `ys` with `push!` rather than with a list comprehension, because we are still building the habit of explicit loops. Adding the first computed point once more closes the polygon; without that step the last side is missing and the shape looks like an open path.